

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

ФАКУЛЬТЕТ РАДИОФИЗИКИ И КОМПЬЮТЕРНЫХ
ТЕХНОЛОГИЙ

Кафедра информатики и компьютерных систем

ДИПЛОМНАЯ РАБОТА

«Разработка комплекса средств для анализа вычислительных
возможностей микропроцессорных систем»

Пашука Тимофея Андреевича
обучающегося 4 курса специальности
«Компьютерная безопасность»

Научный руководитель:
Старший преподаватель
Дрозд Александр Николаевич

Минск, 2026

Оглавление

Введение	3
1 Аналитический обзор и теоретическая база	11
1.1 Постановка задачи системного анализа вычислительной плат- формы	11
1.2 Классы метрик и источники вариативности измерений	14
1.3 Обзор существующих инструментов и подходов	18
2 Архитектурный анализ и проектирование системы анализа	22
2.1 Теоретические предпосылки и требования к системе анализа . .	22
2.1.1 Функциональные требования	24
2.1.2 Нефункциональные требования	26
2.2 Проектирование архитектуры и выбор технологического стека .	27
2.3 Потоки данных и модель обновления интерфейса	29
2.4 Унификация измерений и расширяемость каталога тестов	31
3 Реализация и анализ эффективности программного комплек- са	33
3.1 Реализация модулей мониторинга, тестирования и интерфейса .	33
3.1.1 Подсистема мониторинга: сбор и нормализация метрик .	34
3.1.2 Подсистема бенчмарков: контракт, каталог и механизм запуска	36
3.1.3 Статистическая обработка результатов и устойчивость к шуму	37
3.1.4 Графический интерфейс и конкурентность	38
3.1.5 Кросс-платформенные особенности	40
3.2 Интерпретация результатов и практические рекомендации . . .	41

4	Экспериментальная оценка и анализ результатов	43
4.1	Цель и принципы проведения экспериментов	43
4.2	Методика и условия выполнения	45
4.3	Контроль факторов, влияющих на корректность сравнения . . .	47
4.4	Формат представления результатов	48
4.5	Результаты и сравнение платформ	49
4.5.1	Процессорные тесты	50
4.5.2	Тесты памяти	51
4.5.3	Математические тесты	51
4.5.4	Криптографические тесты	52
4.6	Выводы по экспериментальной части	53
	Заключение	55
	Список использованных источников	61
A	Листинг точки входа приложения (cmd/analyzer/main.go)	65
B	Листинг обработчика измерений (pkg/benchmark/runner.go)	67

Введение

Современный этап развития вычислительной техники характеризуется не только последовательным увеличением номинальной производительности процессоров и подсистем памяти, но и заметным усложнением внутренней организации вычислительной платформы. Если ранее оценка возможностей компьютера нередко сводилась к сравнению тактовой частоты, числа ядер или объёма оперативной памяти, то в настоящее время такого описания уже недостаточно. Реальная производительность системы формируется как результат взаимодействия множества компонентов: микроархитектуры центрального процессора, иерархии кэш-памяти, контроллера оперативной памяти, характеристик накопителя, особенностей сетевой подсистемы, политики планировщика операционной системы, фоновой активности служб, а также используемых программных библиотек и алгоритмов. Вследствие этого одна и та же аппаратная конфигурация способна демонстрировать существенно различающееся поведение в зависимости от характера нагрузки, условий эксплуатации и текущего состояния программной среды.

Особенно важным становится тот факт, что вычислительные системы общего назначения всё чаще используются в условиях многопрофильной нагрузки. На одной и той же рабочей станции или сервере могут одновременно выполняться компиляция программ, запуск виртуальных сред, обработка больших объёмов данных, криптографические операции, сетевое взаимодействие и фоновый мониторинг. В такой ситуации возникает практическая потребность не просто в регистрации отдельных метрик, а в их комплексной интерпретации. Пользователю требуется понимать не только «сколько процентов загрузки показывает CPU», но и почему наблюдается именно такая картина, какие компоненты становятся ограничивающими, насколько устойчивы результаты измерений и как связать полученные показатели с реальными прикладными сценариями.

Актуальность темы исследования определяется сразу несколькими обстоятельствами. Во-первых, для разработчиков программного обеспечения и системных инженеров критично иметь инструмент, позволяющий быстро выявлять узкие места вычислительной платформы без необходимости обращаться к большому числу разрозненных утилит. Во-вторых, в образовательной и исследовательской практике возрастает интерес к наглядным средствам демонстрации влияния архитектурных особенностей процессора, памяти и подсистемы ввода-вывода на итоговую производительность. В-третьих, в задачах компьютерной безопасности и защищённой обработки данных всё большее значение имеет оценка криптографической мощности системы, поскольку скорость хеширования, симметричного шифрования и асимметрических операций напрямую влияет на производительность защищённых протоколов, систем аутентификации, средств контроля целостности и сервисов безопасного хранения данных.

Следует учитывать, что традиционные средства операционной системы, такие как диспетчеры задач, системные мониторы и стандартные консольные утилиты, как правило, ориентированы на отображение состояния «в моменте». Они позволяют получить полезные сведения о текущем потреблении ресурсов, однако редко дают целостное представление о вычислительных возможностях платформы. Например, общая загрузка процессора не отражает распределение нагрузки по логическим ядрам и не позволяет судить о характере параллелизма. Метрики памяти часто показывают лишь степень занятости ресурсов, но не дают ответа на вопрос о пропускной способности или латентности доступа. Дисковые и сетевые показатели обычно фиксируются как мгновенные скорости, однако без сопоставления с фоновыми процессами и режимом работы системы их трудно интерпретировать. Аналогично, отдельные бенчмарки позволяют получить итоговый числовой результат, но нередко не объясняют, в каких условиях он получен и как он соотносится с текущим состоянием системы.

Тем самым возникает противоречие между потребностью в комплексном, воспроизводимом и интерпретируемом анализе вычислительной платформы и ограниченностью существующих подходов, которые либо сосредоточены на оперативном мониторинге, либо предоставляют синтетическую оценку без достаточного контекста. Разрешение этого противоречия предполагает

создание программного комплекса, объединяющего функции телеметрии и измерительного тестирования в рамках единой архитектуры и общего пользовательского интерфейса. Такой подход позволяет одновременно наблюдать состояние системы, инициировать контролируемую нагрузку и интерпретировать результаты с учётом фоновых факторов, что существенно повышает прикладную ценность анализа.

В данной дипломной работе рассматривается и реализуется программный комплекс «Системный анализатор», предназначенный для исследования вычислительных возможностей микропроцессорных систем общего назначения. Концептуальной основой комплекса является сочетание двух взаимодополняющих режимов работы. Первый режим связан с мониторингом телеметрии в реальном времени и обеспечивает получение информации о ключевых системных характеристиках: параметрах CPU, использовании памяти, активности дисков и сети, средней нагрузке, времени работы системы и перечне наиболее ресурсоёмких процессов. Второй режим представляет собой набор синтетических тестов, сгруппированных по прикладным категориям и предназначенных для оценки предельных или характерных возможностей платформы при вычислительных, память-зависимых, математических и криптографических нагрузках.

Принципиально важным является то, что в рамках предлагаемого подхода мониторинг и бенчмарки не рассматриваются как независимые подсистемы. Напротив, они образуют единую информационную среду анализа. Это означает, что пользователь может сопоставлять результат измерительного теста с текущим фоном системы: активностью других процессов, распределением нагрузки по ядрам, состоянием ввода-вывода и иными факторами, способными исказить измерения. Тем самым повышается интерпретируемость результатов и уменьшается риск ошибочных выводов, возникающих при использовании «изолированных» числовых оценок без контекста.

С научно-практической точки зрения задача анализа вычислительной платформы осложняется многомерностью самого понятия производительности. Для процессорных нагрузок существенны не только количество ядер и частота, но и глубина конвейера, эффективность предсказания ветвлений, работа кэшей, поддержка SIMD-инструкций, механизмы энергосбережения и качество распараллеливания. Для памяти важны одновременно объём, пропускная

способность и латентность случайного доступа. Для подсистемы ввода-вывода решающую роль играют характеристики накопителя, тип интерфейса, наличие очередей операций и влияние фоновых служб. Для криптографических задач необходимо учитывать наличие аппаратного ускорения, особенности реализации библиотек и различие в вычислительной природе симметричных, асимметричных и хеш-функциональных алгоритмов. Следовательно, универсальный инструмент анализа должен поддерживать несколько классов измерений и обеспечивать согласованный способ их представления.

Отдельное место в работе занимает проблема воспроизводимости результатов. В многозадачной операционной системе любые синтетические измерения подвержены влиянию случайных возмущений: миграции потоков между ядрами, фоновых обращений к диску, кратковременной сетевой активности, изменения частотных режимов процессора, эффектов прогрева кэшей и иных факторов. Поэтому получение «одного числа» без статистической обработки не может считаться достаточной основой для корректного вывода. В связи с этим в проекте используется подход, предусматривающий разогрев, серию повторных измерений, фильтрацию выбросов и вычисление устойчивых статистических показателей. Такая организация эксперимента позволяет сделать результаты не только более надёжными, но и более полезными в прикладном и исследовательском отношении.

С точки зрения программной реализации важным требованием является кросс-платформенность и практическая применимость решения. Инструмент анализа должен быть пригоден для использования на современных персональных компьютерах и рабочих станциях без сложной настройки, сохранять отзывчивость интерфейса во время выполнения тестов и при этом позволять расширять каталог измерений по мере развития проекта. В данной работе для реализации выбран язык программирования Go, сочетающий нативную компиляцию, развитую стандартную библиотеку, удобные средства конкурентного выполнения и хорошие возможности для системного программирования. Для построения графического интерфейса применяется библиотека Fyne, обеспечивающая единый API для разных операционных систем и достаточную гибкость при создании вкладок, панелей и интерактивных элементов управления.

Архитектурно программный комплекс разделён на несколько логически самостоятельных модулей. Подсистема `pkg/profiling` отвечает за сбор и

агрегирование телеметрии операционной системы, включая сведения о процессоре, памяти, дисках, сети, времени работы, средней нагрузке и активных процессах. Подсистема `pkg/benchmark` реализует каталог измерительных тестов, их унифицированный интерфейс, механизм запуска и статистическую обработку результатов. Подсистема `pkg/gui` обеспечивает пользовательское представление данных, организацию вкладок, визуализацию состояния системы и сценарии запуска тестов, в том числе интерактивные криптографические измерения с изменяемыми параметрами. Подобное разбиение не только повышает сопровождаемость кода, но и соответствует методологической задаче отделения получения данных, вычислительной логики и пользовательского представления.

Следует подчеркнуть, что объектом внимания в работе является не абстрактный «идеальный» бенчмарк, а инструмент, ориентированный на практику системного анализа. Это означает, что при проектировании учитываются не только требования к точности измерений, но и требования к интерпретируемости, удобству использования и связи результатов с прикладными сценариями. Пользователю важно не просто узнать, что та или иная система показывает определённое число мегабайт в секунду или операций в секунду, а понять, что именно означает данный показатель, при каких предпосылках он получен, насколько он стабилен и какие реальные выводы можно из него сделать относительно пригодности платформы для разработки, анализа данных, криптографической обработки или эксплуатации ресурсоёмких сервисов.

Цель, задачи и практическая значимость

Целью дипломной работы является разработка, теоретическое обоснование и практическая реализация программного комплекса для комплексного анализа вычислительных возможностей микропроцессорных систем, обеспечивающего одновременное получение телеметрии в реальном времени и воспроизводимых результатов синтетического тестирования по нескольким категориям нагрузок.

Для достижения поставленной цели в работе решаются следующие **задачи**:

1. Выполнить анализ предметной области и обосновать необходимость объединения средств мониторинга и бенчмаркинга в рамках единого

инструмента системного анализа.

2. Исследовать ключевые классы метрик, характеризующих вычислительную платформу, включая процессорные, память-зависимые, дисковые, сетевые и криптографические показатели.
3. Сформулировать функциональные и нефункциональные требования к программному комплексу с учётом сценариев практической эксплуатации: первичной диагностики, сравнения платформ, учебного применения и анализа аппаратных ограничений.
4. Спроектировать модульную архитектуру приложения и выбрать технологический стек, обеспечивающий кросс-платформенность, расширяемость, достаточную производительность измерений и отзывчивость пользовательского интерфейса.
5. Реализовать подсистему телеметрии, включающую сбор сведений о процессоре, памяти, дисковой и сетевой активности, времени работы системы, средней нагрузке и списке активных процессов.
6. Реализовать библиотеку синтетических тестов по категориям «Процессор», «Память», «Математика», «Криптография» и обеспечить единый механизм их запуска и интерпретации.
7. Разработать подход к повышению устойчивости результатов, включающий разогрев, многократные прогоны, фильтрацию выбросов и вычисление робастных статистических показателей.
8. Создать графический интерфейс, позволяющий пользователю наблюдать состояние системы, запускать тесты, анализировать результаты и выполнять интерактивные криптографические измерения с изменяемыми параметрами.
9. Провести анализ применимости разработанного комплекса, описать ограничения методики измерений и сформулировать рекомендации по интерпретации получаемых результатов.

Объект исследования — вычислительные системы общего назначения, включающие многоядерные микропроцессоры, иерархию памяти, типовые

подсистемы ввода-вывода и программную среду, определяющую условия выполнения прикладных задач.

Предмет исследования — методы программного мониторинга и синтетического тестирования вычислительной платформы, способы повышения воспроизводимости результатов, а также подходы к представлению и интерпретации измеряемых показателей в задачах системного анализа и компьютерной безопасности.

Практическая значимость работы заключается в создании прикладного программного инструмента, который может быть использован:

- для первичной диагностики производительности и текущего состояния рабочей станции, ноутбука или сервера перед развёртыванием ресурсоёмких приложений и сервисов;
- для сравнительного анализа аппаратных платформ по различным профилям нагрузок, включая вычисления общего назначения, операции с памятью, ввод-вывод и криптографическую обработку;
- для выявления факторов нестабильности измерений, обусловленных фоновой активностью, конкуренцией за ресурсы и особенностями планировщика операционной системы;
- в учебном процессе как наглядное средство демонстрации взаимосвязи между архитектурой вычислительной системы, состоянием программной среды и наблюдаемыми характеристиками производительности;
- в задачах предварительной оценки пригодности вычислительной платформы для приложений, чувствительных к пропускной способности памяти, дисковой активности или криптографической нагрузке.

Научная новизна и методическая ценность работы в прикладном контексте заключаются в том, что анализ вычислительной платформы рассматривается не как сумма разрозненных метрик, а как единый процесс сопоставления телеметрии и результатов контролируемых измерений. Подобный подход позволяет перейти от простого наблюдения к осмысленной интерпретации, что особенно важно в условиях многозадачных систем, где числовой результат без контекста может быть недостаточно информативным или даже вводящим в заблуждение.

Структура работы отражает логику исследования. Во введении обосновываются актуальность темы, цель, задачи, объект и предмет исследования, а также практическая значимость работы. В первой главе рассматриваются теоретические основы системного анализа вычислительных платформ и анализ существующих подходов. Во второй главе формулируются требования к комплексу и описываются архитектурные решения, положенные в основу программной реализации. В третьей главе анализируются особенности реализации модулей мониторинга, библиотеки тестов, статистической обработки результатов и пользовательского интерфейса. В четвёртой главе описывается методика проведения экспериментов и подход к анализу результатов. В заключении подводятся итоги выполненной работы и намечаются направления дальнейшего развития проекта.

Таким образом, разработка программного комплекса «Системный анализатор» представляет собой актуальную и практически значимую задачу, находящуюся на пересечении системного программирования, анализа производительности, прикладного бенчмаркинга и компьютерной безопасности. Решение данной задачи позволяет получить инструмент, который не только фиксирует параметры работы системы, но и помогает интерпретировать их в контексте реальных вычислительных сценариев, что делает результаты анализа более содержательными, воспроизводимыми и полезными для пользователя.

Глава 1

Аналитический обзор и теоретическая база

1.1 Постановка задачи системного анализа вычислительной платформы

Под системным анализом вычислительной платформы в рамках данной работы понимается совокупность методов, моделей и практических процедур, направленных на получение воспроизводимого, интерпретируемого и прикладно значимого представления о состоянии аппаратно-программного комплекса и о его предельных возможностях при различных профилях нагрузки. В отличие от локального профилирования отдельного приложения, где исследователь сосредоточен на поведении конкретного алгоритма или программного модуля, системный анализ охватывает платформу в целом. Это означает необходимость рассматривать не только центральный процессор, но и память, дисковую и сетевую подсистемы, а также влияние операционной системы, фоновых процессов, механизмов энергосбережения и особенностей используемой программной среды.

Необходимость такого подхода обусловлена тем, что современная вычислительная система представляет собой сложную многоуровневую структуру, в которой итоговая производительность определяется не отдельным компонентом, а согласованностью взаимодействия всех подсистем. Даже высокопроизводительный процессор не гарантирует высокой прикладной эффективности, если приложение ограничено пропускной способностью памяти, скоростью на-

копителя, сетевой задержкой или особенностями планировщика. Аналогично, высокие паспортные характеристики оборудования не всегда отражают поведение системы в реальных условиях эксплуатации, где присутствуют фоновые службы, антивирусная активность, индексация файлов, обновления, виртуализация и иные процессы, способные существенно изменить измеряемый результат.

В теоретическом плане задача системного анализа тесно связана с понятием производительности как многомерной характеристики. Под производительностью нельзя понимать исключительно скорость выполнения одной инструкции, одного потока или одной операции. Для прикладного анализа существенны по меньшей мере следующие аспекты: вычислительная мощность процессора, способность платформы эффективно использовать параллелизм, скорость и латентность обращения к памяти, интенсивность и устойчивость ввода-вывода, а также производительность криптографических примитивов, если система применяется в задачах защиты информации. Следовательно, корректный анализ должен учитывать разнообразие классов нагрузок и невозможность свести состояние платформы к одному универсальному числу.

С практической точки зрения системный анализ востребован в ряде типовых сценариев. Во-первых, он необходим для первичной диагностики рабочих станций и серверов, когда требуется установить, какие именно ресурсы ограничивают производительность прикладных программ. Во-вторых, он важен для сравнения аппаратных платформ при выборе оборудования под определённый класс задач, например под компиляцию, обработку больших массивов данных, криптографические сервисы или нагрузку, чувствительную к памяти. В-третьих, системный анализ имеет значимость в образовательном и исследовательском процессе, поскольку позволяет наглядно показать, каким образом архитектурные особенности вычислительной системы проявляются в измеряемых показателях.

Отдельно следует отметить, что в условиях современных гетерогенных и многоядерных архитектур интерпретация результатов становится значительно сложнее, чем в эпоху относительно однородных процессоров. Наличие производительных и энергоэффективных ядер, многоуровневых кэшей, аппаратных блоков ускорения и развитых механизмов динамического изменения частоты приводит к тому, что поведение системы начинает существенно зависеть от

характера нагрузки и политики управления ресурсами. По этой причине результаты, полученные без учёта контекста исполнения, могут быть формально корректными, но недостаточно информативными для практических выводов.

Ещё одной принципиальной сложностью является стохастическая природа наблюдаемых измерений. В многозадачной операционной системе любое тестирование выполняется на фоне иных процессов и событий, часть которых не контролируется исследователем напрямую. На результат способны влиять миграция потоков между ядрами, прерывания от устройств, обращение к файловой системе, состояние кэшей, колебания частоты CPU, прогрев компонентов и сетевые события. Это означает, что системный анализ должен включать не только сбор значений, но и механизмы повышения устойчивости выводов: повторные измерения, предварительный разогрев, фильтрацию выбросов и оценку разброса.

В рамках настоящей работы применяется комбинированный подход к решению данной задачи. С одной стороны, используется мониторинг состояния системы в реальном времени, позволяющий зафиксировать контекст выполнения измерений: загрузку процессора, распределение нагрузки по ядрам, использование памяти, дисковую и сетевую активность, а также присутствие конкурирующих процессов. С другой стороны, используются синтетические тесты, создающие контролируемую нагрузку и позволяющие оценить возможности платформы в воспроизводимой форме. Именно сочетание этих двух компонентов делает анализ более содержательным: мониторинг отвечает на вопрос о текущем состоянии системы, а бенчмарки — на вопрос о её поведении при специально организованных вычислительных сценариях.

Таким образом, постановка задачи системного анализа вычислительной платформы в данной работе сводится к созданию такого подхода и такого программного инструмента, которые позволяют одновременно наблюдать состояние системы, формировать репрезентативную нагрузку, сопоставлять измерения с фоновыми факторами и получать результаты, пригодные для практической интерпретации. Это и определяет логику дальнейшего исследования, в котором рассматриваются классы метрик, существующие подходы и программные средства, а также формулируются требования к разрабатываемому комплексу.

1.2 Классы метрик и источники вариативности измерений

Для практического анализа вычислительной платформы целесообразно выделять несколько групп метрик, каждая из которых характеризует отдельный аспект функционирования системы. Подобная классификация важна не только для упорядочивания наблюдаемых данных, но и для корректной интерпретации результатов, поскольку разные классы нагрузок опираются на разные аппаратные и программные ресурсы. Следовательно, методика анализа должна учитывать различную природу вычислительных, память-зависимых, дисковых, сетевых и криптографических процессов.

Процессорные метрики отражают способность платформы выполнять вычисления и эффективно использовать имеющийся параллелизм. К данной группе относятся общая загрузка CPU, загрузка по отдельным логическим ядрам, доступные частотные режимы, средняя нагрузка системы, число физических и логических ядер, а также косвенные признаки конкуренции за процессорное время. Однако интерпретация этих данных требует осторожности. Например, высокая суммарная загрузка CPU не всегда означает, что приложение эффективно использует ресурсы: она может быть вызвана конкуренцией нескольких процессов, неудачным распараллеливанием или повышенной активностью фоновых служб. Напротив, умеренная загрузка может сопровождаться реальным дефицитом вычислительных ресурсов, если потоки часто блокируются или ожидают данных из памяти и ввода-вывода. Поэтому при анализе процессорных показателей важно учитывать не только абсолютный уровень загрузки, но и форму распределения нагрузки по ядрам, динамику частоты, устойчивость режима исполнения и характер выполняемой работы.

С инженерной точки зрения процессорные метрики особенно значимы для задач, чувствительных к интенсивности вычислений, ветвлениям и масштабированию по потокам. К ним относятся компиляция, численные методы, обработка мультимедийных данных, моделирование и часть криптографических операций. Для таких сценариев важны не только средние значения загрузки, но и способность системы длительно поддерживать вычислительную активность без деградации из-за троттлинга, ограничений энергопотребления

или неэффективного планирования. Именно поэтому в прикладных средствах анализа полезно сопоставлять данные о загрузке ядер с результатами синтетических CPU-бенчмарков.

Метрики памяти образуют вторую принципиально важную группу, поскольку для многих современных приложений именно подсистема памяти становится фактическим ограничителем производительности. К этим метрикам относятся общий объём оперативной памяти, объём доступной и занятой памяти, использование подкачки, косвенные признаки memory pressure, а также показатели, характеризующие качество доступа к данным. В прикладном отношении особенно значимы два параметра: пропускная способность и латентность. Пропускная способность показывает, насколько быстро система способна обрабатывать последовательные потоки данных, тогда как латентность определяет время реакции на нерегулярные обращения к памяти. Для задач потоковой обработки, мультимедиа, больших матричных вычислений и копирования массивов критична именно пропускная способность. Для компиляторов, графовых алгоритмов, баз данных и многих аналитических приложений большое значение имеет латентность случайного доступа.

Особенность памяти как объекта анализа заключается в том, что её влияние на производительность часто носит скрытый характер. Приложение может демонстрировать внешне умеренную загрузку CPU, но фактически быть ограниченным не вычислениями как таковыми, а задержками получения данных из кэша более низкого уровня или из основной памяти. По этой причине измерение только объёма занятой памяти не даёт достаточной информации для системного анализа. Требуются дополнительные тесты, позволяющие оценить скорость линейных операций и чувствительность платформы к нерегулярному шаблону обращений.

Метрики подсистемы ввода-вывода включают показатели работы накопителей и сетевых интерфейсов. Для дисковой подсистемы важны скорости чтения и записи, интенсивность операций, характер доступа к данным, влияние буферизации и наличие конкурирующей фоновой активности. Для сетевой подсистемы существенны скорости передачи и приёма данных, наличие активных соединений и общая интенсивность сетевого обмена. Следует учитывать, что для многих приложений ввод-вывод выступает не просто вспомогательным компонентом, а фактором, определяющим предельную производительность.

Это особенно характерно для резервного копирования, работы с большими файлами, систем сборки, обработки журналов, потоковой передачи данных и распределённых сервисов.

Системный анализ ввода-вывода осложняется тем, что наблюдаемые скорости сильно зависят от текущего состояния очередей операций, кэширования со стороны ОС, параллельной активности других программ и особенностей файловой системы. Например, высокие показатели чтения могут объясняться не только быстрым накопителем, но и попаданием данных в системный кэш. Аналогично, всплеск записи может быть связан с временной буферизацией, а не с устойчивой физической скоростью устройства. Следовательно, для интерпретации I/O-метрик необходим контекст в виде дополнительной телеметрии и повторных измерений.

Криптографические метрики имеют особое значение в рассматриваемой работе, поскольку значительная часть современных приложений функционирует в среде, где защита данных является обязательным требованием. К этой группе относятся пропускная способность симметричного шифрования, скорость вычисления хеш-функций, производительность асимметрических операций и эффективность алгоритмов аутентифицированного шифрования. На практике именно эти показатели определяют, насколько быстро система может обслуживать защищённые соединения, выполнять цифровую подпись, проверять целостность файлов и обрабатывать конфиденциальные данные.

Криптографические метрики сложны для анализа по нескольким причинам. Во-первых, разные алгоритмы по своей природе нагружают различные элементы вычислительной платформы: одни зависят преимущественно от арифметической мощности и ширины векторных инструкций, другие — от производительности операций с большими целыми числами, третьи — от особенностей доступа к памяти. Во-вторых, результат существенно зависит от того, поддерживает ли платформа специализированные аппаратные инструкции ускорения. В-третьих, сами криптографические библиотеки могут быть реализованы по-разному, что делает интерпретацию абсолютных чисел без знания программного контекста неполной. Тем не менее именно включение криптографических тестов в состав общего системного анализа делает инструмент особенно полезным для задач компьютерной безопасности.

Наряду с перечисленными группами метрик следует учитывать и **кон-**

текстные системные показатели, которые не являются целевым объектом бенчмаркинга, но оказывают прямое влияние на достоверность выводов. К ним можно отнести время работы системы, наличие активных фоновых процессов, интенсивность обслуживания диска и сети, а также общий уровень загрузки ОС. Эти данные помогают объяснить причины нестабильности результатов и позволяют отделить особенности аппаратной платформы от случайных внешних воздействий.

Важнейшей проблемой при работе с любыми перечисленными метриками является вариативность измерений. Она обусловлена прежде всего тем, что современные операционные системы работают в многозадачном режиме и постоянно перераспределяют ресурсы между конкурентными задачами. Даже при неизменной конфигурации оборудования последовательные измерения могут различаться из-за миграции потоков между ядрами, обслуживания прерываний, подкачки памяти, фоновой индексации файлов, сетевой активности, изменений температурного режима или динамического регулирования частоты процессора. Следовательно, единичное измерение не всегда отражает типичное поведение платформы.

Дополнительным источником вариативности являются эффекты прогрева и предварительного состояния системы. Первые итерации теста могут выполняться в условиях холодных кэшей, неинициализированных структур данных и ещё не стабилизировавшихся частотных режимов. В последующих итерациях результаты зачастую улучшаются или, напротив, ухудшаются, если сказывается нагрев и ограничения энергопотребления. Кроме того, при измерении коротких операций существенную роль может играть накладная стоимость самой инфраструктуры тестирования, поэтому методика должна учитывать достаточное время прогона и осмысленное число повторений.

В связи с этим в данной работе используется подход, при котором измерения организуются как серия испытаний с предварительным разогревом и последующей статистической обработкой. Разогрев позволяет уменьшить влияние неустоявшегося начального состояния. Повторные прогоны дают материал для оценки разброса. Фильтрация выбросов снижает искажение результатов редкими внешними событиями, а использование устойчивых статистических характеристик делает итоговые значения более надёжными. Таким образом, анализ метрик в настоящем исследовании строится не на единичных наблюде-

ниях, а на совокупности данных, интерпретируемых в контексте состояния всей системы.

1.3 Обзор существующих инструментов и подходов

Практика анализа вычислительных систем традиционно основывается на сочетании нескольких классов программных средств, каждое из которых решает собственную задачу. В общем виде такие средства можно разделить на встроенные системные мониторы, синтетические бенчмарки общего назначения, специализированные тесты отдельных подсистем и профилировщики, ориентированные на разработчика. Несмотря на их полезность, ни один из этих классов в отдельности не обеспечивает одновременно полноту контекста, воспроизводимость измерений и удобство прикладной интерпретации результатов.

Первую группу составляют **встроенные средства операционной системы**. К ним относятся диспетчеры задач, мониторы ресурсов, системные панели наблюдения и консольные утилиты, показывающие загрузку CPU, использование памяти, активность дисков и сети. Преимуществом таких средств является их доступность, оперативность и непосредственная привязка к текущему состоянию платформы. Они незаменимы для диагностики уже наблюдаемой проблемы: например, при резком росте загрузки процессора, нехватке памяти или аномальной дисковой активности. Однако данные инструменты, как правило, ориентированы на моментальный срез состояния и не предназначены для выполнения воспроизводимых серий измерений. Они не формируют контролируемую нагрузку и не дают единого механизма сопоставления разных классов показателей.

Ограниченность встроенных средств особенно заметна в тех случаях, когда требуется не просто зафиксировать факт высокой нагрузки, а ответить на вопрос о предельных возможностях системы. Например, монитор ресурсов может показать текущее потребление CPU или RAM, но не определит пропускную способность памяти при потоковой обработке данных, не оценит латентность случайного доступа и не покажет, каковы возможности платформы в криптографических вычислениях. Кроме того, встроенные инструменты редко

дают пользователю научно или методически оформленную интерпретацию результатов, оставляя выводы на усмотрение оператора.

Вторую группу образуют **синтетические бенчмарки общего назначения**. Наиболее характерный пример такого подхода представляют популярные кросс-платформенные тестовые пакеты, ориентированные на быстрое сравнение систем по набору стандартных нагрузок. Их основное достоинство заключается в удобстве: пользователь получает сводный результат или набор оценок, на основании которых можно сопоставлять разные устройства между собой. Подобные инструменты особенно востребованы в прикладной практике, где необходимо быстро оценить относительный уровень производительности без развёрнутой аналитики.

Однако универсальные бенчмарки общего назначения обладают и существенными ограничениями. Во-первых, они часто функционируют как «чёрный ящик»: пользователь видит итоговый балл или обобщённый набор чисел, но не всегда имеет доступ к деталям методики, параметрам теста и механизму интерпретации. Во-вторых, такие средства, как правило, слабо учитывают текущее состояние системы в момент измерения. Они сообщают результат, но не объясняют, в какой степени на него повлияли фоновые процессы, загрузка диска, нестабильность частоты или иные внешние факторы. В-третьих, универсальные пакеты не всегда позволяют адаптировать тестирование под прикладной профиль пользователя, что ограничивает их ценность при решении конкретных инженерных задач.

Третью группу составляют **специализированные бенчмарки отдельных подсистем**. Сюда относятся средства оценки производительности процессора, памяти, дисков, сети или конкретных криптографических алгоритмов. Их сильной стороной является фокусировка на одном классе задач, что позволяет реализовать более точную и содержательную методику измерения. Например, тесты памяти могут отдельно оценивать пропускную способность и задержки, дисковые тесты — различать последовательный и случайный доступ, а криптографические утилиты — измерять скорость конкретного алгоритма в конкретном режиме работы.

Недостаток специализированных средств состоит в их фрагментарности. Пользователь вынужден обращаться к нескольким независимым программам, каждая из которых использует собственный формат вывода, собственный

набор параметров и собственную методику представления результатов. Это затрудняет сопоставление показателей между собой и усложняет формирование целостной картины состояния платформы. Кроме того, раздельное использование множества инструментов не всегда позволяет связать результат отдельного теста с общим состоянием системы в момент выполнения измерения.

Четвёртую группу представляют **профилировщики и инструменты разработчика**, например семейство `pprof` в экосистеме Go. Эти средства ориентированы прежде всего на анализ поведения конкретного программного продукта: выявление горячих участков кода, оценку распределения времени выполнения, анализ использования памяти, блокировок и активности горутин. Их значение для оптимизации прикладного ПО чрезвычайно велико, поскольку они позволяют установить, какие именно функции и структуры данных формируют основную нагрузку.

Тем не менее профилировщики не решают задачу первичного системного анализа платформы как таковой. Они требуют наличия исследуемого приложения и репрезентативного сценария его выполнения. Получаемые результаты носят прикладной, а не платформенный характер: например, высокий процент времени в определённой функции объясняет поведение данной программы, но не даёт полноценного ответа на вопрос о предельной пропускной способности памяти системы в целом или о сравнительной эффективности разных аппаратных платформ без привязки к конкретному приложению.

С точки зрения задач компьютерной безопасности следует также учитывать наличие **криптографически ориентированных средств тестирования**. Они полезны для измерения скорости хеширования, шифрования, генерации ключей и иных операций, однако в большинстве случаев остаются узкоспециализированными утилитами. Такие средства позволяют оценить конкретные алгоритмы, но, как правило, не связывают криптографические показатели с общей телеметрией системы и не встраиваются в более широкий контекст анализа вычислительной платформы.

Обобщая рассмотренные классы инструментов, можно сделать вывод, что существующие подходы либо хорошо решают задачу наблюдения, либо хорошо решают задачу измерения, но редко объединяют их в единой методически целостной форме. Встроенные мониторы дают контекст, но не обеспечивают

контролируемого тестирования. Универсальные бенчмарки дают быстрый результат, но часто недостаточно прозрачны и слабо связаны с текущим состоянием системы. Специализированные тесты точны, но фрагментарны. Профилировщики глубоки, но ориентированы на отдельные программы, а не на платформу как целое.

Именно поэтому в рамках настоящей работы обосновывается подход «единое окно», в котором мониторинг, контекстная телеметрия и синтетические бенчмарки интегрируются в одном программном комплексе. С научно-методической точки зрения это позволяет перейти от набора разрозненных показателей к согласованной модели анализа. С практической точки зрения такой подход упрощает сравнение платформ, повышает интерпретируемость результатов и делает инструмент полезным как для инженерной диагностики, так и для учебного применения.

Следовательно, разработка собственного программного комплекса системного анализа представляется обоснованной не как дублирование уже существующих средств, а как попытка объединить их сильные стороны и устранить наиболее существенные ограничения. В дальнейшем это позволяет сформулировать требования к архитектуре системы, способу представления данных и методике выполнения измерений, что и рассматривается в последующих главах работы.

Глава 2

Архитектурный анализ и проектирование системы анализа

2.1 Теоретические предпосылки и требования к системе анализа

Разработка прикладного инструмента для анализа вычислительных возможностей вычислительной платформы требует исходить из того, что производительность не является простой и однозначной характеристикой. На уровне пользовательского восприятия компьютер может рассматриваться как единое устройство, однако на уровне инженерного анализа его поведение определяется взаимодействием нескольких неоднородных подсистем. Итог любой вычислительной нагрузки зависит от эффективности выполнения инструкций в ядре процессора, степени использования параллелизма, скорости и латентности доступа к памяти, состояния дисковой и сетевой подсистем, а также от фоновых воздействий операционной системы и активных процессов. Следовательно, проектирование системы анализа не может ограничиваться регистрацией одной-двух агрегированных метрик; необходимо формировать архитектуру, ориентированную на комплексный сбор, интерпретацию и сопоставление данных разных типов.

Теоретической предпосылкой проектирования является признание того

факта, что для разных классов приложений «узкое место» формируется по-разному. Для одних задач критична вычислительная плотность и способность системы поддерживать высокий темп арифметических операций. Для других определяющим фактором становится память: либо из-за ограниченной пропускной способности при потоковой обработке данных, либо из-за высокой латентности случайного доступа. Для третьих нагрузок основным ограничением оказывается подсистема ввода-вывода, если приложение активно взаимодействует с накопителем, сетью или внешними устройствами. По этой причине архитектура инструмента анализа должна обеспечивать возможность наблюдать не только итоговый результат теста, но и тот контекст, в котором этот результат был получен.

Для вычислительных нагрузок критичен параллелизм как на уровне инструкций, так и на уровне потоков исполнения. Современные процессоры используют внеочередное исполнение, конвейеризацию, предсказание ветвлений, многоуровневые кэши и, во многих случаях, гетерогенную организацию ядер. В таких условиях одна и та же программа может демонстрировать существенно различающуюся эффективность в зависимости от структуры кода, числа потоков, локальности доступа к данным и поведения планировщика. Именно поэтому в проектируемом комплексе предусмотрены тесты, формирующие различный профиль процессорной нагрузки: сортировка большого массива данных нагружает кэш-подсистему и предсказатель ветвлений; рекурсивные вычисления акцентируют накладные расходы вызовов функций, работу со стеком и чувствительность к последовательной логике; параллельные вычисления по методу Монте-Карло позволяют оценивать масштабируемость по числу доступных ядер и эффективность конкурентного исполнения.

Подсистема памяти выступает самостоятельным источником ограничений, который часто проявляется менее очевидно, чем недостаток вычислительной мощности. Во многих приложениях процессор простаивает не потому, что не способен выполнять операции достаточно быстро, а потому, что ожидает поступления данных из кэша более низкого уровня или из основной памяти. По этой причине в проекте реализованы два принципиально разных класса измерений памяти: оценка пропускной способности линейных операций и оценка латентности нерегулярных обращений. Первый тип измерений отражает возможности контроллера памяти и общую эффективность работы с

последовательными массивами данных. Второй тип ориентирован на имитацию практических сценариев со слабой пространственной локальностью, где используется случайная цепочка индексов по принципу pointer-chasing, снижающая влияние аппаратной предвыборки и позволяющая приблизиться к более реалистичной оценке задержки доступа.

В контексте компьютерной безопасности существенную роль играет и криптографическая производительность. Современные системы постоянно выполняют шифрование, хеширование, аутентификацию, проверку цифровых подписей и генерацию ключевого материала. Следовательно, оценка вычислительной платформы без учёта криптографических операций была бы неполной, особенно если предполагается использование системы в сетевых сервисах, системах защищённого хранения данных или инфраструктуре аутентификации. В проекте поэтому выделен отдельный набор криптографических измерений. С одной стороны, алгоритмы хеширования, такие как SHA-256 и SHA-512, характеризуют скорость проверки целостности и обработки потоков данных. С другой стороны, симметричные и асимметричные алгоритмы, включая AES, RSA, Ed25519 и ChaCha20-Poly1305, позволяют судить о пропускной способности защищённых каналов и стоимости операций, характерных для протоколов безопасности. Для этой категории в системе реализованы как автоматические тесты, так и интерактивный режим, позволяющий изменять параметры и наблюдать, как они влияют на скорость.

Таким образом, уже на этапе формулирования требований становится очевидным, что разрабатываемая система анализа должна опираться на комбинированный подход. С одной стороны, она должна собирать и отображать телеметрию в реальном времени, фиксируя состояние платформы «здесь и сейчас». С другой стороны, она должна обеспечивать воспроизводимые измерения с использованием синтетических тестов, нацеленных на разные классы нагрузок. Только при объединении этих двух составляющих пользователь получает возможность не просто наблюдать показатели, а интерпретировать их с учётом причин, условий и ограничений.

2.1.1 Функциональные требования

С точки зрения пользовательских сценариев комплекс должен обеспечивать несколько взаимосвязанных групп функций. Первая группа связана с на-

блюдением за текущим состоянием системы. Пользователь должен получать информацию о загрузке процессора, распределении активности по логическим ядрам, состоянии памяти, дисковой и сетевой активности, времени работы системы, средней нагрузке и перечне процессов, формирующих наибольшее потребление ресурсов. При этом отображение должно быть организовано таким образом, чтобы метрики воспринимались не как набор изолированных чисел, а как связанная картина состояния платформы.

Вторая группа функций относится к синтетическому тестированию. Система должна позволять запускать измерительные сценарии по нескольким категориям нагрузок, соответствующим ключевым подсистемам вычислительной платформы. Пользователь должен иметь возможность инициировать тест, получить итоговый результат в естественных прикладных единицах измерения, ознакомиться с пояснением смысла полученного числа и сопоставить его с качеством измерения. Последний аспект является особенно важным, поскольку без информации о разбросе, выбросах или коэффициенте вариации числовой результат остаётся потенциально неоднозначным.

Третья группа функций связана с интерпретацией и сопровождением результата. Пользовательский интерфейс должен не только выводить итоговое значение, но и пояснять, как трактовать его с прикладной точки зрения. Например, высокая пропускная способность памяти должна быть понятна не сама по себе, а как признак эффективности платформы для потоковой обработки данных. Аналогично, производительность криптографических операций должна сопоставляться с типовыми сценариями: скоростью шифрования трафика, интенсивностью хеширования больших файлов или возможным числом асимметричных операций в единицу времени.

Отдельно выделяется требование интерактивного режима криптографических измерений. Пользователь должен иметь возможность выбирать алгоритм, конфигурацию ключа, режим работы и объём данных. Система при этом обязана обеспечивать корректное измерение времени выполнения, учитывать различие между короткими и длинными операциями, а также выводить результат в наглядной форме. Подобная функциональность делает комплекс не только средством наблюдения, но и инструментом исследовательского характера, позволяющим анализировать влияние параметров алгоритма на вычислительные затраты.

Наконец, функциональные требования включают поддержку расширения каталога тестов без переработки всего приложения. Это означает, что добавление нового измерения не должно требовать изменения базовой логики визуализации, механизма запуска или структуры всего интерфейса. Такая постановка задачи непосредственно влияет на архитектурные решения, рассматриваемые далее.

2.1.2 Нефункциональные требования

К ключевым нефункциональным требованиям относятся кросс-платформенность, отзывчивость интерфейса, воспроизводимость результатов, расширяемость и модульность архитектуры. Кросс-платформенность важна по нескольким причинам. Во-первых, она позволяет использовать один и тот же инструмент для анализа различных операционных систем, что существенно повышает прикладную ценность комплекса. Во-вторых, она делает возможным сравнение платформ в сопоставимой программной среде. В-третьих, она снижает стоимость сопровождения, так как вместо нескольких отдельных приложений поддерживается единая кодовая база.

Отзывчивость интерфейса означает, что пользовательский UI не должен блокироваться ни при выполнении серии бенчмарков, ни при периодическом обновлении мониторинговых виджетов. Для инструмента анализа это требование является не косметическим, а принципиальным: если интерфейс теряет отзывчивость во время теста, пользователь лишается возможности наблюдать контекст измерений и взаимодействовать с системой. Следовательно, архитектура должна предусматривать асинхронное выполнение длительных операций и безопасную передачу результатов в слой представления.

Воспроизводимость подразумевает устойчивость результатов к фоновым факторам и осмысленную организацию методики измерений. Она достигается не только наличием повторных прогонов, но и поддержкой разогрева, фильтрации выбросов, оценкой разброса и использованием единых правил расчёта итогового показателя. Без этих механизмов инструмент был бы пригоден лишь для ориентировочной оценки, но не для сравнительного анализа или учебно-исследовательской работы.

Расширяемость и модульность имеют особое значение, поскольку предметная область анализа вычислительных систем постоянно развивается. Появля-

ются новые классы нагрузок, новые криптографические алгоритмы, новые метрики телеметрии и новые сценарии интерпретации результатов. Поэтому набор тестов и отображаемых показателей должен расширяться без изменения базовой архитектуры и без необходимости полностью перерабатывать интерфейс при добавлении нового измерения. Иначе проект быстро утратит сопровождаемость и станет плохо пригоден для дальнейшего развития.

Кроме того, к нефункциональным требованиям можно отнести интерпретируемость, то есть способность системы представлять данные в форме, понятной пользователю. Для прикладного инструмента мало просто собрать показатели; необходимо так организовать их отображение, чтобы пользователь мог связать число с реальным поведением платформы и понять, как использовать этот результат в инженерной или учебной практике.

2.2 Проектирование архитектуры и выбор технологического стека

Проектирование архитектуры программного комплекса начинается с выбора технологического стека, который должен удовлетворять как функциональным, так и нефункциональным требованиям. В данной работе программный комплекс реализован на языке Go, что обусловлено сочетанием практических преимуществ для задач системного анализа. Компиляция в нативный код обеспечивает предсказуемую производительность синтетических тестов и снижает влияние исполняющей среды на характер нагрузки. Это особенно важно для бенчмарков, где нежелательны дополнительные уровни абстракции, способные вносить нестабильность или трудноучитываемые накладные расходы.

Существенным преимуществом Go является развитая стандартная библиотека, включающая основные криптографические примитивы, средства работы со временем, конкурентным исполнением, буферами данных и системным взаимодействием. Для рассматриваемого проекта это означает возможность строить измерительные сценарии на базе хорошо поддерживаемых и переносимых компонентов без чрезмерной зависимости от внешних пакетов. Дополнительным аргументом в пользу Go является модель конкурентности, основанная на горутинах и каналах. Она позволяет естественным образом разделять фоновый сбор телеметрии, выполнение тестов и обновление ин-

терфейса, не прибегая к избыточно сложным механизмам синхронизации на уровне операционной системы.

Для графической части выбрана библиотека Fyne, обеспечивающая единый API для разных платформ и достаточную скорость отрисовки при регулярном обновлении виджетов. В контексте данного проекта выбор GUI-библиотеки определяется не только удобством создания окна и стандартных элементов управления, но и возможностью быстро формировать вкладки, карточки, таблицы, прогресс-индикаторы и интерактивные формы. Поскольку приложение ориентировано на регулярное обновление данных мониторинга, библиотека интерфейса должна выдерживать такой режим без заметного ухудшения пользовательского опыта.

Архитектура построена модульно и отражает границы ответственности:

- **pkg/profiling** — сбор телеметрии ОС. Модуль получает информацию о модели и характеристиках процессора, использовании памяти, дисковом и сетевом вводе-выводе, средней нагрузке (load average), состоянии дискового пространства, а также формирует список наиболее активных процессов. Для гетерогенных процессоров Apple Silicon дополнительно реализована попытка определения количества P- и E-ядер через вызовы `sysctl`.
- **pkg/benchmark** — библиотека тестов и ядро запуска. Все тесты приводятся к единому интерфейсу `BenchmarkFunc`, что позволяет расширять набор измерений без изменения внешней логики. Запуск осуществляется через `TestRunner`, который выполняет серию итераций, выполняет «разогрев» для стабилизации состояния, фильтрует выбросы методом IQR и вычисляет робастные показатели (медиана, MAD, коэффициент вариации). Данная схема уменьшает влияние фоновых задач ОС и делает вывод более воспроизводимым.
- **pkg/gui** — представление данных и управление сценариями пользователя. Точка входа приложения формирует вкладки: «Мониторинг», «Процессор», «Память», «Математика», «Криптография». Панель мониторинга отображает телеметрию с периодом обновления 1 секунду и визуализирует нагрузку по каждому логическому ядру. Панели бенчмарков запускают тесты по выбранной категории и показывают интерпретацию

результата в прикладных единицах.

Такое разбиение не является случайным; оно отражает принцип разделения ответственности между слоями приложения. Подсистема телеметрии отвечает за получение и предварительную нормализацию данных. Подсистема бенчмарков концентрирует измерительную логику и статистическую обработку. Подсистема GUI отвечает за сценарии взаимодействия с пользователем и визуализацию. Благодаря этому каждый слой остаётся относительно независимым: изменение способов отображения не требует вмешательства в алгоритмы тестирования, а расширение набора метрик не должно приводить к переработке механизмов статистического расчёта.

Подобная архитектура позволяет также изолировать измерительную нагрузку от UI: вычисления выполняются в фоновых горютинах, а интерфейс получает только готовые значения или сообщения о прогрессе. Это поддерживает отзывчивость даже во время серии тестов и уменьшает риск того, что само средство отображения станет значимым источником искажения результатов. Одновременно модульность задаёт понятный путь расширения: новые тесты добавляются как функции в `pkg/benchmark`, а в UI подключаются через описание задачи и интерпретацию результата без изменения архитектурных слоёв.

2.3 Потоки данных и модель обновления интерфейса

Проектирование комплекса требует явного разделения данных, которые обновляются периодически, и данных, вычисляемых по запросу пользователя. Мониторинговые метрики формируют непрерывный поток обновлений с фиксированным шагом по времени. Они отражают текущее состояние системы и должны отображаться в виде текущих значений, индикаторов и сводных характеристик. Бенчмарки, напротив, запускаются событийно и представляют собой ограниченную по времени серию вычислений, результат которой должен быть представлен пользователю как завершённый измерительный акт вместе со статистикой качества и пояснением смысла показателя.

Такое различие в природе данных непосредственно влияет на архитектуру

потоков. Если обращаться с мониторинговыми метриками и результатами тестов одинаково, система либо станет избыточно сложной, либо начнёт смешивать короткоживущие и непрерывные данные в одном интерфейсном контуре. Поэтому в приложении принята модель, при которой сбор телеметрии и выполнение бенчмарков существуют как разные процессы взаимодействия с системой, но сходятся на уровне пользовательского представления.

В практической реализации это означает, что мониторинг выполняется в фоне с предсказуемой периодичностью, а результаты тестов вычисляются в отдельных фоновых задачах. Интерфейс не занимается «сырым» измерением и не должен принимать участие в тяжёлых вычислениях. Его функция заключается в том, чтобы получать готовые агрегированные значения, отображать прогресс, предоставлять пользователю средства запуска сценариев и сохранять визуальную целостность данных.

Подобная модель снижает взаимное влияние мониторинга и измерений. Во-первых, мониторинг остаётся достаточно редким и стабильным, чтобы не искажать результаты тестов чрезмерной собственной активностью. Во-вторых, длительные бенчмарки не блокируют отрисовку интерфейса и не лишают пользователя возможности наблюдать состояние системы во время прогона. В-третьих, сохраняется важный контекст: в момент измерения пользователь может сопоставить итоговый результат с фоновой нагрузкой, активными процессами и поведением системы в реальном времени.

С архитектурной точки зрения можно говорить о двух типах потоков данных: телеметрических и событийно-измерительных. Телеметрические потоки ориентированы на регулярность, малую задержку отображения и устойчивую нормализацию показателей. Событийно-измерительные потоки ориентированы на корректность расчёта, повторяемость методики и на атомарное представление завершённого результата. Разведение этих потоков по логическим уровням позволяет сделать систему одновременно информативной, отзывчивой и устойчивой к росту функциональности.

2.4 Унификация измерений и расширяемость каталога тестов

Для того чтобы пользовательский интерфейс мог одинаково работать с результатами разнородных тестов, все измерения приводятся к единому контракту: тест возвращает числовой показатель и единицу измерения. На первый взгляд такое решение выглядит простым, однако именно оно задаёт важнейшее архитектурное свойство всей системы — отделение вычислительной части от способа представления результата. Интерфейс не должен знать внутреннюю специфику каждого алгоритма, объём выделяемой памяти, число внутренних итераций или конкретную реализацию теста. Ему достаточно получить числовое значение, единицу измерения и при необходимости связанную функцию интерпретации.

Унификация результатов даёт сразу несколько преимуществ. Во-первых, она снижает связанность между модулем тестирования и GUI, что положительно влияет на сопровождаемость кода. Во-вторых, она упрощает добавление новых тестов: разработчику достаточно реализовать функцию измерения, встроить её в каталог задач и определить способ интерпретации результата. В-третьих, она делает возможным единый механизм статистической обработки, так как все измерения проходят через общую инфраструктуру запуска и агрегации.

Расширяемость каталога тестов особенно важна для предметной области системного анализа, поскольку перечень интересующих пользователя измерений не является фиксированным. Сегодня акцент может быть сделан на CPU, память и криптографию, а в дальнейшем могут потребоваться дополнительные сценарии: файловые профили, сетевые испытания, анализ латентности конкретных структур данных, тесты устойчивости многопоточного выполнения или оценки специализированных ускорителей. Если архитектура не допускает такого роста без значительной переработки, инструмент быстро теряет практическую ценность.

В рассматриваемом проекте добавление нового теста сводится к реализации функции, регистрации задачи в каталоге и подключению описания, необходимого для пользователя. При этом не требуется изменение механизма запуска, дублирование логики статистической обработки или пересборка

всей модели представления. Такой подход соответствует принципу архитектурной открытости для расширения и закрытости для модификации, что особенно важно для исследовательских и учебных программных комплексов, развивающихся по мере накопления новых требований.

Следовательно, унификация измерений и продуманная расширяемость каталога тестов выступают не частной технической деталью, а одним из центральных архитектурных решений проекта. Именно они позволяют сохранить логическую целостность системы при росте числа сценариев анализа и обеспечивают возможность дальнейшего развития комплекса без утраты его сопровождаемости и интерпретируемости.

Глава 3

Реализация и анализ эффективности программного комплекса

3.1 Реализация модулей мониторинга, тестирования и интерфейса

Реализация комплекса построена вокруг идеи «одного окна» для системного анализа, сформулированной на этапе проектирования архитектуры. На практике это означает, что пользователь должен иметь возможность одновременно наблюдать текущее состояние системы, запускать измерительные сценарии и интерпретировать результаты в одном и том же приложении, не переходя между несколькими разрозненными инструментами. С инженерной точки зрения такая постановка задачи существенно усложняет реализацию, поскольку мониторинг и тестирование конкурируют за вычислительные ресурсы, а сам пользовательский интерфейс при неудачной организации может стать дополнительным источником искажения измерений.

Если вычислительные тесты выполняются в том же потоке, что и отрисовка графического интерфейса, приложение быстро теряет отзывчивость, пользователь лишается обратной связи о ходе измерения, а мониторинг перестаёт быть полезным контекстом. Если же телеметрия обновляется слишком часто или избыточно тяжёлым способом, она начинает оказывать заметное влияние

на результаты бенчмарков, особенно на короткие тесты, чувствительные к системным возмущениям. Следовательно, реализация комплекса должна была решить две взаимосвязанные задачи: изолировать измерительную логику от слоя представления и одновременно обеспечить их координацию в рамках единой интерактивной среды.

В результате программный комплекс был реализован как совокупность модулей, взаимодействующих через понятные и достаточно узкие интерфейсы. Подсистема мониторинга отвечает за периодический сбор и предварительную нормализацию метрик, подсистема бенчмарков — за выполнение тестов и статистическую агрегацию результатов, а графический слой — за запуск сценариев, отображение текущего состояния и интерпретацию итоговых значений. Такая схема обеспечивает не только функциональную полноту приложения, но и его сопровождаемость: логика получения данных, вычислительная нагрузка и пользовательское представление не смешиваются в одном уровне абстракции.

3.1.1 Подсистема мониторинга: сбор и нормализация метрик

Сбор телеметрии. Модуль `pkg/profiling` реализует набор функций получения метрик преимущественно на базе библиотеки `gopsutil`, которая предоставляет переносимые интерфейсы для доступа к системной информации. Для центрального процессора выполняется получение сведений о модели, частотных характеристиках и текущей загрузке каждого логического ядра. Последняя измеряется с использованием `cpu.Percent` на интервале порядка одной секунды. Выбор именно такого окна наблюдения обусловлен компромиссом между оперативностью и устойчивостью: слишком короткий интервал давал бы излишне шумные значения, тогда как слишком длинный ухудшал бы восприятие динамики нагрузки в интерфейсе.

Для подсистемы памяти используется `mem.VirtualMemory`, что позволяет получить сведения о доступной, используемой и общей памяти, а также о ряде вспомогательных характеристик, связанных с состоянием памяти на конкретной платформе. Для дисковой подсистемы применяется агрегирование счётчиков `disk.IOCounters` и вычисление скоростей чтения и записи как

разности между последовательными измерениями. Аналогичный принцип используется для сети через `net.IOCounters`, где интерес представляют не абсолютные накопленные счётчики, а текущая интенсивность обмена. Такой подход переводит технические системные величины в форму, более понятную пользователю и пригодную для интерпретации в контексте прикладной нагрузки.

Отдельно реализован сбор общей системной информации: времени работы системы, версии платформы, а также средней нагрузки (`load average`) там, где она доступна. Показатель средней нагрузки дополняет проценты загрузки CPU и позволяет оценивать не только факт использования процессорного времени, но и степень насыщения планировщика. Если значения нагрузки стабильно превышают число логических процессоров, это может свидетельствовать о наличии очереди готовых к выполнению задач и о дефиците вычислительных ресурсов при текущем профиле работы.

Важной практической частью мониторинга является показ «топ-процессов». В модуле `pkg/profiling/processes.go` формируется список процессов, для которых запрашиваются показатели использования CPU и памяти. Затем результаты сортируются и представляются пользователю в текстовой таблице фиксированной ширины. Несмотря на внешнюю простоту, данная функция играет существенную роль в интерпретации измерений. Она позволяет не просто увидеть, что система нагружена, а понять, какими именно процессами формируется эта нагрузка. Например, нестабильный результат теста может быть объяснён работой компилятора, индексацией IDE, фоновым обновлением ОС или активностью браузера.

С инженерной точки зрения подсистема мониторинга решает не только задачу получения метрик, но и задачу их нормализации. Пользователю важны не внутренние системные счётчики сами по себе, а прикладные показатели в естественных единицах измерения. Поэтому модуль преобразует агрегированные значения в формы, пригодные для непосредственного восприятия: для диска и сети — в скорости обмена, для CPU — в проценты загрузки по ядрам, для памяти — в объёмные характеристики с интерпретируемым отношением занятости и доступности ресурсов.

Кроме того, при реализации мониторинга учитывалась необходимость сохранения умеренной собственной нагрузки инструмента. Если телеметрия

будет собираться чрезмерно часто или с чрезмерной детализацией, комплекс начнёт влиять на систему не только как наблюдатель, но и как активный потребитель ресурсов. Поэтому период обновления и состав отображаемых данных были выбраны так, чтобы обеспечить полезную информативность без заметного ухудшения чистоты измерений.

3.1.2 Подсистема бенчмарков: контракт, каталог и механизм запуска

Организация запуска бенчмарков. Подсистема тестирования построена на принципе унифицированного контракта. Все тесты приводятся к единому интерфейсу `BenchmarkFunc`, возвращающему числовой показатель и строку единицы измерения. Такое решение имеет фундаментальное значение для всей системы, так как позволяет абстрагировать пользовательский интерфейс от внутренней специфики конкретного алгоритма. Панель отображения результатов не обязана знать, что именно выполнялось внутри теста: сортировка, шифрование, вычисление хеша или операция с памятью. Ей достаточно получить итоговое значение и единицу измерения, после чего результат может быть представлен в общей форме и дополнен интерпретацией.

В графическом слое каждая задача описывается структурой `BenchmarkTask`, включающей название, категорию, краткое и научное описание, функцию интерпретации результата и ссылку на сам тест. Благодаря этому набор бенчмарков превращается в формализованный каталог, который можно расширять без изменения базового механизма отображения. С архитектурной точки зрения это позволяет отделить «что измеряется» от «как показывается», а с практической — упрощает развитие комплекса и снижает стоимость добавления новых сценариев.

В рамках реализации особое значение имеет разделение тестов по категориям. Пользователь воспринимает измерения не как набор низкоуровневых функций, а как группы задач: «Процессор», «Память», «Математика», «Криптография». Такой подход решает сразу две задачи. Во-первых, он уменьшает когнитивную нагрузку, позволяя запускать измерения в понятных прикладных терминах. Во-вторых, он упрощает последующий анализ, поскольку результаты уже сгруппированы по профилю нагрузки и могут быть сопоставлены

внутри своей функциональной области.

С инженерной точки зрения каталог бенчмарков представляет собой промежуточный слой между вычислительной логикой и интерфейсом. Он позволяет описывать тест не только как функцию, но и как пользовательский сценарий с определённым смыслом. Благодаря этому система приобретает свойства, характерные не просто для набора утилит, а для целостного аналитического инструмента.

3.1.3 Статистическая обработка результатов и устойчивость к шуму

Особое внимание в реализации уделено статистике, так как измерения выполняются в условиях многозадачной операционной системы и неизбежно подвержены шуму. Даже если тестовая функция детерминирована, фактическое время её выполнения зависит от множества внешних факторов: миграции потоков между ядрами, фоновых прерываний, активности других процессов, изменений частоты процессора, прогрева кэшей и иных воздействий. По этой причине единичный запуск не может рассматриваться как достаточная основа для надёжного вывода.

TestRunner использует следующую схему:

1. выполняется разогрев, чтобы исключить влияние ленивой инициализации и прогрева кэшей;
2. выполняется серия итераций (минимум 10), результаты сортируются;
3. применяется IQR-фильтрация для удаления выбросов, возникающих из-за планировщика, фоновых прерываний и коротких всплесков активности других процессов;
4. рассчитываются робастные показатели: медиана, MAD и коэффициент вариации, после чего итоговое значение формируется как взвешенная комбинация медианы и среднего.

Разогрев необходим для того, чтобы исключить влияние неустоявшегося начального состояния. В первых итерациях часть времени может тратиться на выделение памяти, инициализацию структур данных, наполнение кэшей и

стабилизацию режима исполнения. Если эти значения включить в итог без поправок, они могут исказить реальную оценку устойчивой производительности. Серия повторных измерений, в свою очередь, даёт возможность оценить разброс и выявить нестабильность, связанную с внешними факторами.

Использование IQR-фильтрации позволяет отбрасывать редкие выбросы, возникающие, например, при внезапной активности планировщика или кратковременном вмешательстве фоновой задачи. Такой подход особенно полезен в интерактивном приложении, где важно получать практично устойчивые результаты без необходимости переходить к сложным статистическим моделям, избыточным для повседневного системного анализа.

Коэффициент вариации, выраженный в процентах, выводится пользователю как индикатор качества измерения. Чем выше разброс, тем менее надёжны единичные значения и тем сильнее влияние внешних факторов. Такая интерпретация особенно важна при сравнении устройств, поиске регресса производительности и оценке воспроизводимости эксперимента. Пользователь получает не только итоговый показатель, но и информацию о том, насколько этому показателю можно доверять в текущих условиях.

Выбранная методика удобна тем, что сохраняет баланс между практичностью и строгостью. С одной стороны, измерения выполняются достаточно быстро для интерактивного применения и не превращают запуск теста в длительную лабораторную процедуру. С другой стороны, пользователь получает не просто «одно число», а набор статистически осмысленных характеристик, достаточных для принятия решения о повторном прогоне, изменении условий эксперимента или сравнении результатов между платформами.

3.1.4 Графический интерфейс и конкурентность

Графический интерфейс. Точка входа приложения (`cmd/analyzer/main.go`) создаёт окно и набор вкладок: «Мониторинг», «Процессор», «Память», «Математика», «Криптография». Такая организация пространства интерфейса отражает принятую на этапе проектирования классификацию сценариев анализа. Пользователь получает не перегруженное единое полотно, а структурированную среду, в которой каждой группе задач соответствует собственная логическая область.

Вкладка мониторинга построена на панели `NewDashboardPanel`, внутри

которой в фоне выполняется цикл с периодом около одной секунды, собирающий метрики и обновляющий виджеты. Визуализация загрузки по ядрам реализована через набор индикаторов `ProgressBar`, создаваемых динамически под фактическое число логических процессоров. Такой подход позволяет адаптировать интерфейс к разным платформам без жёстко зафиксированного числа ядер и одновременно даёт пользователю наглядное представление о распределении нагрузки, а не только о её усреднённом значении.

Для запуска измерений используются панели `NewBenchmarkPanel(category)`, которые инкапсулируют логику работы с каталогом тестов в конкретной категории. Панель отвечает за запуск серии измерений, ожидание результата, отображение прогресса и выдачу итогового значения вместе с интерпретацией. Подобное решение обеспечивает единообразие пользовательского опыта: независимо от категории нагрузок пользователь взаимодействует с логически одинаковыми элементами интерфейса.

Отдельным элементом является интерактивная криптопанель `NewInteractiveCryptoPanel`. Она позволяет выбирать алгоритм, параметры ключа или хеша, а также объём тестовых данных. Включение такой панели в состав комплекса имеет особую ценность, поскольку переводит приложение из режима простого «запуска предопределённых тестов» в режим исследовательского взаимодействия с вычислительной системой. Пользователь может варьировать параметры и непосредственно наблюдать, как эти изменения отражаются на временных затратах и пропускной способности.

Для больших объёмов данных применяется сэмплирование и экстраполяция времени, что делает интерфейс практичным и предотвращает длительную блокировку пользователя при попытке выполнить крайне тяжёлую операцию в тестовом режиме. С инженерной точки зрения это пример компромисса между точностью и удобством: приложение сохраняет возможность оценивать масштаб поведения алгоритма без необходимости реально выполнять заведомо чрезмерный объём работы.

Для обеспечения отзывчивости интерфейса все потенциально длительные операции, включая серии бенчмарков и криптографические измерения на больших объёмах, выполняются в фоновых горутинах. UI получает обновления

по прогрессу и итоговым значениям, что позволяет одновременно наблюдать систему и контролировать ход измерения. Такое разделение особенно важно в режимах, где тест выполняет десятки итераций, а мониторинг параллельно обновляет виджеты с фиксированным периодом. Без подобной конкурентной организации приложение быстро стало бы непрактичным для реального использования.

3.1.5 Кросс-платформенные особенности

В ходе реализации учитываются различия платформ и доступность метрик на разных операционных системах. Хотя библиотека `gopsutil` и абстрагирует значительную часть платформенно-зависимых деталей, отдельные показатели всё же могут быть недоступны, интерпретироваться по-разному или предоставляться с различной степенью детализации. Это касается, например, средней нагрузки, некоторых счётчиков ввода-вывода, информации о процессах и специфичных характеристик аппаратной платформы.

В таких ситуациях комплекс придерживается принципа деградации функциональности. Если определённая метрика не может быть получена корректно, пользователь продолжает получать остальную доступную информацию, а сама система не прекращает работу и не нарушает выполнение измерительных сценариев. Такой подход особенно важен для прикладного инструмента, который должен быть полезен в разных средах, даже если они предоставляют неидентичный объём телеметрии.

Отдельного упоминания заслуживает поддержка гетерогенных архитектур, в частности платформ семейства Apple Silicon. Для них в проекте предпринята попытка различать производительные и энергоэффективные ядра через системные вызовы `sysctl`. Даже если такая информация не всегда доступна в полном объёме, сама поддержка подобного сценария подчёркивает ориентацию комплекса на актуальные архитектурные особенности современных вычислительных систем.

3.2 Интерпретация результатов и практические рекомендации

Результаты синтетических тестов в комплексе формируются в прикладных величинах, которые проще соотнести с реальными задачами. Именно этот принцип интерпретации был одним из ключевых при разработке всей системы. Пользователь получает не абстрактный балл, а значение в естественной единице измерения, из которой можно сделать вывод о практической пригодности платформы. Например, пропускная способность AES в МБ/с напрямую отражает потенциальную скорость шифрования трафика в VPN, системе защищённого хранения данных или сервере приложений. Показатель RSA-операций в секунду соотносится с числом возможных асимметричных операций, характерных для аутентификации и рукопожатий TLS. Для хеширования показатель МБ/с показывает, насколько быстро система способна проверять целостность больших архивов, образов, резервных копий или потоков данных.

Схожим образом интерпретируются и другие категории. Процессорные тесты позволяют судить о способности платформы эффективно справляться с вычислительными нагрузками, чувствительными к ветвлениям, рекурсии и масштабированию по потокам. Тесты памяти отражают качество работы с большими линейными массивами и нерегулярными структурами данных. Математические бенчмарки показывают общую эффективность выполнения численных операций и могут быть соотнесены с задачами моделирования, аналитики и научных вычислений.

Для процессорных тестов важна интерпретация не только абсолютного значения, но и стабильности результата. Если коэффициент вариации высок, это означает, что в системе присутствует переменная нагрузка или нестабильный режим исполнения, вмешивающийся в измерения. В таком случае для корректного сравнения платформ рекомендуется:

- закрыть ресурсоёмкие приложения и повторить серию измерений;
- сопоставить результаты с «топ-процессами» и показателями дискового/сетевого ввода-вывода;
- оценить нагрузку по ядрам: равномерная высокая загрузка подтверждает CPU-bound режим, а неравномерная может означать ограничения по

синхронизации или приоритетам планировщика.

Для подсистемы памяти следует различать ситуации «высокая пропускная способность при высокой латентности» и «низкая пропускная способность при низкой латентности», поскольку в реальных приложениях это приводит к разному поведению. Поточковая обработка данных, мультимедиа и часть научных вычислений чаще зависят от пропускной способности. Базы данных, компиляторы, графовые структуры и индексные хранилища значительно чувствительнее к латентности случайных обращений. Поэтому корректная интерпретация результатов памяти невозможна без понимания прикладного профиля исследуемой задачи.

Показатели дискового и сетевого ввода-вывода также необходимо рассматривать совместно с фоном системы. Высокая скорость чтения или записи не всегда означает устойчиво высокую производительность накопителя: она может быть частично обусловлена системным кэшированием или отсутствием конкурирующих задач в момент измерения. Аналогично, сетевые показатели следует соотносить с внешними условиями и характером текущей активности. Именно поэтому наличие мониторинговой вкладки и списка активных процессов играет не вспомогательную, а методически значимую роль.

В практическом отношении комплекс может использоваться в нескольких сценариях. Он полезен для первичной диагностики рабочей станции или сервера, когда необходимо быстро понять, какой компонент системы ограничивает производительность. Он пригоден для сравнительного анализа аппаратных платформ в учебной и инженерной практике. Наконец, он представляет интерес как демонстрационный инструмент, позволяющий наглядно показать связь между архитектурой вычислительной системы, текущим состоянием ОС и наблюдаемыми результатами синтетических измерений.

Таким образом, реализованный программный комплекс позволяет не только получить числовые показатели, но и связать их с состоянием системы в момент измерения, с качеством статистической устойчивости и с прикладным смыслом результата. Это делает инструмент пригодным как для инженерной диагностики, так и для учебного анализа архитектурных особенностей современных вычислительных платформ и методов программного измерения производительности.

Глава 4

Экспериментальная оценка и анализ результатов

4.1 Цель и принципы проведения экспериментов

Экспериментальная часть работы предназначена для практической проверки пригодности разработанного программного комплекса к задачам системного анализа, а также для демонстрации того, каким образом получаемые измерения могут использоваться при сравнении реальных вычислительных платформ. В отличие от чисто теоретического обсуждения архитектуры приложения, данный раздел ориентирован на подтверждение того, что предложенный подход позволяет формировать интерпретируемые, устойчивые и прикладно значимые результаты в условиях реальных операционных систем и фоновой активности.

Цель экспериментальной части состоит не только в сравнении двух платформ различной архитектуры и программной среды, но и в проверке воспроизводимости измерений комплекса «Системный анализатор», а также в анализе того, насколько удобен разработанный инструмент для практической интерпретации результатов. Иными словами, в рамках экспериментов оценивается не только поведение тестируемого оборудования, но и состоятельность самого подхода, основанного на объединении телеметрии и синтетических бенчмарков в едином интерфейсе.

В качестве тестовых систем рассматриваются:

- MacBook Air 15'' (M2, 2023), macOS Ventura+;
- ASUS Vivobook 15 (Ryzen 5 4600H), Windows 11.

Выбор именно таких платформ обусловлен их различием как по микроархитектуре, так и по программной среде. Одна система относится к семейству современных ARM-совместимых энергоэффективных платформ Apple Silicon, вторая — к x86-64 платформам массового сегмента на базе мобильного процессора AMD Ryzen. Такое сопоставление представляет интерес не только с прикладной точки зрения, но и с методической: оно позволяет оценить, насколько разработанный комплекс способен корректно функционировать в неоднородных программно-аппаратных условиях и предоставлять сопоставимый формат результата при наличии различий в операционной системе, механизмах энергосбережения, драйверной модели и наборе доступных метрик.

При интерпретации результатов принципиально учитывается то обстоятельство, что измерения в многозадачной операционной системе неизбежно содержат шум. Любой запуск теста происходит не в изолированной лабораторной среде, а в системе, где параллельно работают фоновые службы, диспетчеры окон, сетевые стеки, файловые индексы и другие процессы. Для уменьшения влияния случайных факторов в комплексе используются разогрев, серия итераций и робастная статистическая обработка, включающая фильтрацию выбросов и оценку разброса результатов. Дополнительно в ходе эксперимента фиксируются условия выполнения: режим питания, наличие фоновой нагрузки, версия ОС и общая характеристика тестовой конфигурации.

С методической точки зрения эксперименты строятся на нескольких принципах. Во-первых, сравнение должно выполняться не по одному изолированному числу, а по совокупности результатов, характеризующих различные классы нагрузок. Во-вторых, любые числовые показатели должны интерпретироваться с учётом качества измерения и телеметрического контекста. В-третьих, выводы по экспериментальной части должны быть сформулированы не как абстрактное ранжирование платформ, а как анализ сильных и слабых сторон каждой системы применительно к конкретным профилям вычислительных задач.

4.2 Методика и условия выполнения

Корректность экспериментальной оценки в значительной степени зависит от единообразия условий выполнения тестов. Даже при использовании одного и того же программного комплекса результаты могут существенно различаться, если один прогон проводится при высокой фоновой нагрузке, а другой — в близком к «чистому» состоянии системы. Поэтому методика эксперимента включает набор организационных требований, направленных на снижение влияния плохо контролируемых факторов.

Перед запуском измерений рекомендуется:

1. закрыть ресурсоёмкие приложения и дождаться завершения фоновых задач (обновления, индексация);
2. подключить устройство к сети питания (или, если цель — оценка «в полевых условиях», явно зафиксировать режим работы от батареи);
3. зафиксировать параметры системы: модель CPU, объём RAM, объём и тип накопителя, версию ОС;
4. выполнить по крайней мере 3 независимых прогона каждой категории, сравнивая медианные значения и коэффициент вариации.

Каждый из перечисленных пунктов имеет практическое и методическое значение. Заккрытие ресурсоёмких приложений уменьшает вероятность паразитной конкуренции за CPU, память и дисковую подсистему. Подключение к сети питания важно для исключения искажений, связанных с агрессивными профилями энергосбережения, особенно на ноутбуках. Фиксация конфигурации системы необходима для корректного документирования эксперимента и последующего воспроизведения условий. Наконец, выполнение нескольких независимых прогонов позволяет перейти от единичного наблюдения к статистически более осмысленной оценке.

Важным элементом методики является также предварительный анализ состояния системы перед началом каждой серии тестов. Пользователь или исследователь должен учитывать текущую загрузку ядер, наличие активных фоновых процессов, дисковую и сетевую активность, а также общее время работы системы после загрузки. Например, запуск бенчмарка сразу после

включения компьютера может сопровождаться активностью служб инициализации, а проведение измерений после длительной работы под нагрузкой может приводить к иному тепловому режиму и, как следствие, к другим частотным характеристикам процессора.

В рамках дипломной работы результаты представляются в виде таблиц по основным категориям бенчмарков: «Процессор», «Память», «Математика», «Криптография». Отдельно приводятся скриншоты интерфейса комплекса, позволяющие зафиксировать форму представления данных и иллюстрировать, каким образом пользователь наблюдает состояние системы и результаты запусков. Наличие графических иллюстраций важно не только как элемент визуального сопровождения, но и как подтверждение того, что разработанный комплекс действительно обеспечивает единый контур наблюдения и измерения.

На рис. 4.1 показан главный экран комплекса с набором вкладок, а на рис. 4.2 — пример панели тестов памяти, демонстрирующей формат отображения результатов и единицы измерения.

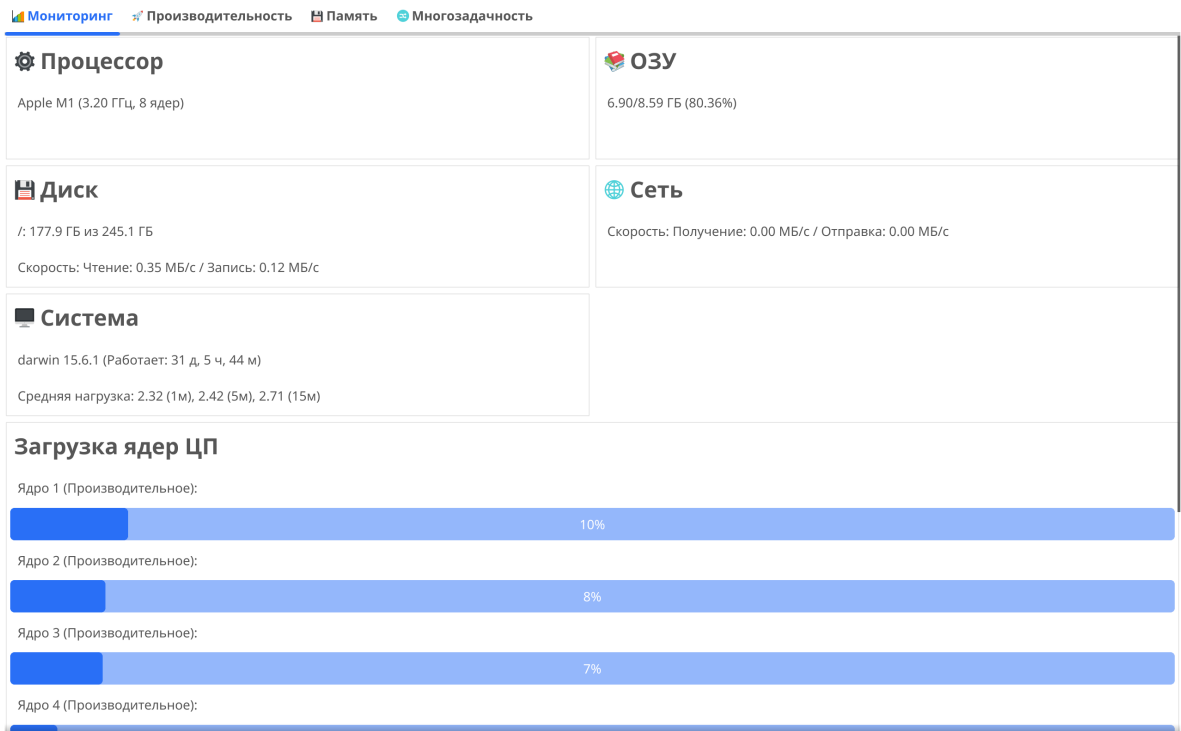


Рис. 4.1: Главное окно комплекса «Системный анализатор»

Представленные скриншоты иллюстрируют, что экспериментальная работа с комплексом не сводится к консольному запуску отдельных утилит. Пользователь получает структурированный графический интерфейс, в котором мониторинг и бенчмарки сосуществуют в едином пространстве. Это особенно

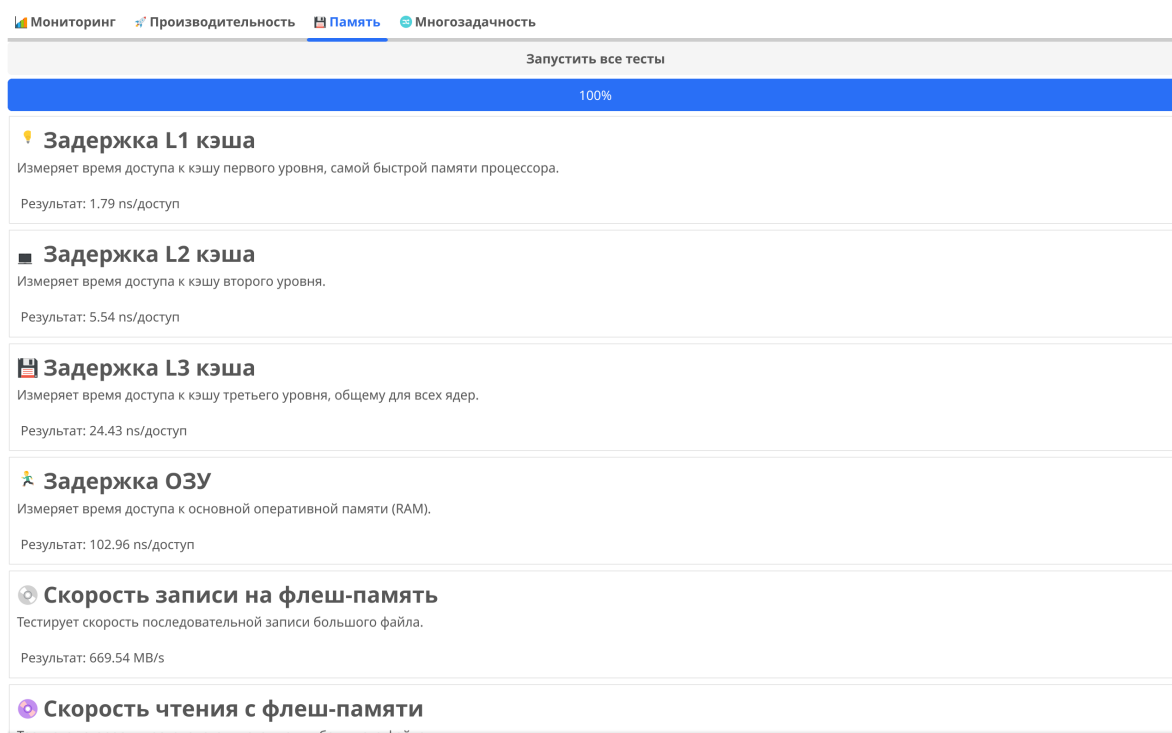


Рис. 4.2: Пример панели тестов памяти и представления результатов

важно для экспериментов, ориентированных не только на получение числовых результатов, но и на интерпретацию условий их формирования.

4.3 Контроль факторов, влияющих на корректность сравнения

Сравнение двух вычислительных платформ всегда требует учёта факторов, которые затрудняют прямую интерпретацию чисел «в лоб». Даже если тестовая нагрузка одинакова на уровне исходного кода, реальное выполнение зависит от множества параметров: операционной системы, профиля энергопотребления, алгоритмов планировщика, поведения подсистемы памяти, драйверной модели и особенностей фоновых служб. Следовательно, любой сравнительный эксперимент должен сопровождаться фиксацией допущений и ограничений.

В рассматриваемой работе особое внимание уделяется следующим потенциальным источникам несопоставимости:

- различие архитектур процессоров и наборов инструкций, что влияет на эффективность криптографических и арифметических операций;

- различие операционных систем, определяющее поведение планировщика, доступность телеметрии и характер фоновой активности;
- различие режимов энергосбережения и тепловых ограничений, особенно значимое для ноутбуков;
- возможное влияние кэширования, предвыборки и фоновых операций ввода-вывода на результаты коротких тестов;
- ограниченная воспроизводимость в условиях реального пользовательского окружения по сравнению с лабораторной стендовой средой.

Учёт перечисленных факторов не устраняет полностью вариативность измерений, но делает анализ более корректным. В частности, это позволяет отделять архитектурные различия платформ от случайных отклонений, связанных с моментным состоянием системы. Именно поэтому в рамках данного исследования результат рассматривается не как абсолютная и окончательная характеристика устройства, а как эмпирическая оценка его поведения в контролируемых, но реалистичных условиях эксплуатации.

4.4 Формат представления результатов

Для обеспечения сопоставимости итогов в работе используются единые правила представления и интерпретации результатов. Такой подход необходим, поскольку сравниваются не только разные платформы, но и разные классы нагрузок, для которых естественные единицы измерения принципиально различаются. Без предварительно заданных правил сравнение чисел разных типов было бы методически некорректным.

В качестве основных принципов принимаются следующие положения:

- для показателей пропускной способности (МБ/с) сравниваются медианные значения;
- для латентности (нс) меньшие значения трактуются как более предпочтительные;
- для операций/с (например, RSA) большее значение соответствует более высокой производительности;

- коэффициент вариации рассматривается как индикатор устойчивости измерения.

Использование медианы в качестве базового сравниваемого показателя оправдано тем, что она менее чувствительна к единичным выбросам, чем среднее арифметическое. Это особенно важно в многозадачной среде, где даже один нехарактерный эпизод фоновой активности может заметно сместить среднее значение. Коэффициент вариации, в свою очередь, позволяет оценить, насколько надёжен наблюдаемый результат: высокая производительность при высоком разбросе не всегда предпочтительнее умеренной, но устойчивой производительности.

Для удобства дальнейшего анализа результаты целесообразно трактовать в двух плоскостях: количественной и качественной. Количественная плоскость отражает собственно числовое преимущество одной платформы над другой. Качественная плоскость показывает, как этот числовой результат соотносится с характером нагрузки и практическими сценариями применения. Такой двухуровневый подход делает экспериментальную часть более содержательной и приближает её к задачам реального системного анализа.

4.5 Результаты и сравнение платформ

Данный раздел предназначен для фиксации результатов экспериментального прогона комплекса на двух целевых платформах. Структурирование по категориям выбрано не случайно: оно соответствует логике самого программного комплекса и позволяет анализировать поведение систем не в виде общего «рейтинга», а по отдельным профилям нагрузки. Такой подход методически предпочтителен, поскольку одна и та же платформа может демонстрировать сильные стороны в одном классе задач и уступать в другом.

Таблицы ниже представляют форму документирования результатов, пригодную для последующего заполнения фактическими измеренными значениями. Важно подчеркнуть, что сами таблицы являются не просто способом визуального оформления, а частью методики: они задают единый формат фиксации результатов, минимизируют риск потери контекста и упрощают аналитическое сравнение после завершения серий прогонов.

4.5.1 Процессорные тесты

Процессорные тесты предназначены для оценки вычислительной мощности платформ, характера масштабирования по потокам и устойчивости поведения при нагрузках, чувствительных к структуре кода. В данной категории особенно важно учитывать не только итоговое значение, но и форму распределения нагрузки по ядрам, а также влияние архитектурных особенностей процессора, включая число производительных и энергоэффективных ядер, особенности кэш-подсистемы и динамическое изменение частот.

При анализе результатов CPU-бенчмарков необходимо обращать внимание на несколько аспектов. Во-первых, следует оценивать абсолютный уровень производительности, выраженный в прикладной единице измерения. Во-вторых, необходимо сопоставлять его со стабильностью результата: высокий разброс может свидетельствовать о сильном влиянии фоновой активности или о неустойчивом частотном режиме. В-третьих, полезно анализировать показания мониторинга в момент теста: равномерная загрузка ядер обычно соответствует корректному распараллеливанию, тогда как выраженная асимметрия может указывать на ограничения со стороны планировщика, синхронизации или архитектуры самой нагрузки.

Таблица 4.1: Сравнение процессорных тестов

Тест	Ед.	MacBook Air M2	Ryzen 5 4600H

После заполнения таблицы интерпретация должна строиться не только на сравнении отдельных строк, но и на анализе общей картины. Если одна платформа демонстрирует преимущество в задачах с высокой долей ветвлений, но уступает в параллельных сценариях, это может указывать на различие в микроархитектурных приоритетах. Если же различия невелики, но коэффициент вариации заметно различается, практический вывод может заключаться не столько в абсолютном превосходстве, сколько в большей предсказуемости поведения одной из систем.

4.5.2 Тесты памяти

Тесты памяти направлены на исследование двух взаимосвязанных, но не тождественных характеристик: пропускной способности и латентности доступа. Эта категория особенно важна, поскольку в современных приложениях ограничение по памяти нередко проявляется скрыто и может ошибочно восприниматься как недостаток процессорной мощности. Сравнение платформ по параметрам памяти позволяет выявить, насколько эффективно каждая из них работает с большими потоками данных и насколько чувствительна к нерегулярным обращениям.

При интерпретации результатов памяти необходимо учитывать прикладной профиль задач. Высокая пропускная способность благоприятна для потоковой обработки, мультимедиа, некоторых научных и матричных вычислений. Низкая латентность особенно важна для сценариев, включающих сложные структуры данных, графы, базы данных, компиляторы и иные нагрузки с большим числом случайных обращений. Поэтому итоговый вывод по данной категории должен учитывать, какая именно характеристика памяти является более значимой для предполагаемого класса задач.

Таблица 4.2: Сравнение тестов памяти

Тест	Ед.	MacBook Air M2	Ryzen 5 4600H

При дальнейшем анализе желательно отдельно обсуждать ситуации, когда одна из платформ выигрывает по пропускной способности, а другая — по латентности. Подобный результат не является противоречием; напротив, он отражает различие в архитектурных приоритетах и может иметь существенные прикладные последствия. Именно поэтому в рамках системного анализа память должна рассматриваться не как «один ресурс», а как совокупность характеристик доступа к данным.

4.5.3 Математические тесты

Математические тесты занимают промежуточное положение между «чисто процессорными» и более специализированными сценариями. С одной стороны, они в значительной степени зависят от общей арифметической мощности

процессора и эффективности компиляции. С другой стороны, они могут демонстрировать чувствительность к организации циклов, локальности данных и особенностям работы с числами с плавающей точкой или целочисленной арифметикой.

В прикладном отношении данная категория важна для оценки пригодности платформы к задачам моделирования, аналитики, статистической обработки и инженерных вычислений. Если в рамках эксперимента наблюдается преимущество одной платформы в математических тестах, это может быть следствием как более высокой вычислительной плотности, так и лучшей адаптации микроархитектуры к соответствующему типу операций. В то же время стабильность результатов здесь не менее важна, чем абсолютные значения, поскольку математические сценарии нередко используются в длинных вычислительных цепочках, где предсказуемость времени исполнения имеет практическое значение.

Таблица 4.3: Сравнение математических тестов

Тест	Ед.	MacBook Air M2	Ryzen 5 4600H

После заполнения данной таблицы интерпретация должна учитывать, насколько обнаруженные различия устойчивы между независимыми прогонами и коррелируют ли они с результатами общих CPU-бенчмарков. Если математические тесты повторяют картину процессорной категории, это усиливает уверенность в выводах. Если же наблюдается иная картина, это может быть поводом для более тонкого анализа архитектурных особенностей платформы.

4.5.4 Криптографические тесты

Криптографические тесты позволяют оценить поведение платформ в задачах, связанных с защитой информации, обработкой конфиденциальных данных и поддержкой защищённых коммуникаций. В отличие от общих CPU-бенчмарков, здесь результаты особенно чувствительны к наличию специализированных инструкций ускорения, качеству реализации библиотек и различию между алгоритмами симметричного, асимметричного и хеш-функционального типов.

Для симметричных алгоритмов ключевой характеристикой обычно является пропускная способность, определяющая потенциальную скорость обработки больших потоков данных. Для асимметричных алгоритмов важнее число операций в секунду, поскольку такие алгоритмы чаще используются в сравнительно коротких, но вычислительно дорогих операциях установления доверия, генерации подписей и проверки ключевого материала. Для хеширования существенна скорость обработки больших массивов данных, напрямую связанная с задачами контроля целостности, индексирования и построения цифровых отпечатков.

Таблица 4.4: Сравнение криптографических тестов

Алгоритм	Ед.	MacBook Air M2	Ryzen 5 4600H

После получения фактических значений интерпретация этой таблицы должна учитывать, что «лучшая» платформа может различаться в зависимости от конкретного алгоритма. Преимущество в симметричном шифровании не гарантирует преимущества в асимметрических операциях, и наоборот. Именно поэтому криптографическая категория особенно показательна для демонстрации того, что универсального однозначного лидера по всем видам вычислительной нагрузки может не существовать.

4.6 Выводы по экспериментальной части

По итогам экспериментов должны быть сформулированы выводы о сравнительной производительности платформ в разрезе категорий нагрузок, а также о стабильности результатов, выдаваемых комплексом. При этом итоговая оценка не должна сводиться к упрощённому утверждению о том, что одна система «лучше» другой по всем параметрам. Гораздо более корректным является профильный вывод: одна платформа может оказаться предпочтительной для криптографических операций, другая — для части вычислительных или память-зависимых задач, третьи особенности могут проявляться в стабильности и предсказуемости результата.

Не менее важным выводом экспериментальной части является подтверждение того, что разработанный комплекс позволяет анализировать платформу

не в отрыве от контекста, а в связи с текущим состоянием системы. Именно наличие мониторинговой телеметрии, списка активных процессов, распределения нагрузки по ядрам и статистических характеристик измерения делает итоговые числа более содержательными и пригодными для инженерной интерпретации.

Отдельно должны фиксироваться ограничения сравнения: различие операционных систем, различие механизмов энергосбережения, особенности микроархитектуры, доступность аппаратных ускорителей, а также невозможность полного устранения внешних факторов в условиях пользовательской среды. Признание этих ограничений не ослабляет научную ценность эксперимента, а, напротив, делает анализ более корректным и методически добросовестным.

Таким образом, экспериментальная часть выступает логическим завершением всей работы: она демонстрирует, что разработанный программный комплекс способен выполнять не только функции визуализации и запуска тестов, но и служить инструментом получения воспроизводимых и интерпретируемых данных о вычислительных возможностях современных микропроцессорных систем.

Заключение

В рамках данной дипломной работы разработан и реализован программный комплекс «Системный анализатор», предназначенный для комплексного анализа вычислительных возможностей микропроцессорных систем общего назначения. Основная идея исследования заключалась в том, что эффективный анализ современной вычислительной платформы не может строиться исключительно на разрозненных средствах мониторинга или на изолированных синтетических тестах. Для получения практически значимого результата требуется единая среда, в которой текущее состояние системы, результаты управляемых измерений и контекст их возникновения могут рассматриваться совместно. Именно такой подход был положен в основу разработанного решения.

Актуальность поставленной задачи определяется тем, что современные вычислительные платформы стали существенно более сложными как на аппаратном, так и на программном уровне. Многоядерность, гетерогенность вычислительных ядер, развитая иерархия памяти, фоновая активность операционной системы, механизмы энергосбережения и наличие аппаратных ускорителей приводят к тому, что производительность перестаёт быть характеристикой, описываемой одним числом. Пользователю, разработчику или системному инженеру необходимо не просто фиксировать загрузку CPU или объём использованной памяти, а понимать, какие именно факторы ограничивают эффективность платформы, насколько устойчивы наблюдаемые результаты и как связать их с реальными прикладными сценариями.

В ходе работы было показано, что системный анализ вычислительной платформы целесообразно рассматривать как совокупность взаимосвязанных процедур наблюдения, измерения и интерпретации. На теоретическом уровне была обоснована необходимость комплексного подхода, сочетающего мониторинг телеметрии в реальном времени и выполнение синтетических бенчмарков

по нескольким категориям нагрузок. На практическом уровне этот подход был воплощён в программном комплексе, объединяющем сбор системной информации, библиотеку тестов, статистическую обработку результатов и графический интерфейс в рамках единой архитектуры.

Итоги выполненного исследования могут быть сформулированы более подробно следующим образом.

1. **Решена задача анализа предметной области и обоснована постановка проблемы.** В работе показано, что существующие средства системного наблюдения, профилирования и бенчмаркинга обладают высокой практической ценностью, но, как правило, решают лишь часть общей задачи. Одни средства дают текущее состояние системы без контролируемого измерительного сценария, другие — предоставляют синтетический результат без достаточного контекста. Это послужило основанием для разработки интегрированного решения, ориентированного на одновременное наблюдение и тестирование.
2. **Исследованы ключевые классы метрик вычислительной платформы.** В работе были рассмотрены процессорные, память-зависимые, дисковые, сетевые и криптографические показатели, а также показано, что каждая из этих групп характеризует отдельный аспект поведения системы. Это позволило сформировать методическую основу для построения каталога тестов и системы интерпретации результатов.
3. **Сформулированы требования к программному комплексу.** На основании анализа предметной области были выделены функциональные требования, связанные с мониторингом состояния системы, запуском измерительных сценариев и интерпретацией результатов, а также нефункциональные требования, включающие кросс-платформенность, отзывчивость пользовательского интерфейса, воспроизводимость измерений, расширяемость и модульность архитектуры.
4. **Спроектирована и реализована модульная архитектура приложения.** Программный комплекс разделён на подсистемы, соответствующие различным областям ответственности: `pkg/profiling` обеспечивает сбор системной телеметрии, `pkg/benchmark` реализует библиотеку измерительных тестов и общий механизм их запуска, `pkg/gui` формирует

пользовательский интерфейс и сценарии взаимодействия с системой. Такое разделение позволило изолировать вычислительную нагрузку от слоя представления и обеспечить логическую целостность проекта.

5. **Реализован мониторинг состояния системы в реальном времени.** Приложение отображает сведения о центральном процессоре, памяти, дисковой и сетевой активности, времени работы системы, средней нагрузке и наиболее ресурсоёмких процессах. Дополнительно визуализируется распределение загрузки по логическим ядрам, что повышает интерпретируемость результатов и помогает связывать поведение бенчмарков с фактическим состоянием платформы.
6. **Разработана библиотека бенчмарков по основным категориям нагрузок.** В пользовательском интерфейсе реализованы вкладки «Процессор», «Память», «Математика», «Криптография», каждая из которых объединяет тесты соответствующего профиля. Такой подход позволяет анализировать не абстрактную «общую производительность», а применимость платформы к различным классам прикладных задач.
7. **Внедрён механизм робастной статистической обработки результатов.** Запуск тестов осуществляется через `TestRunner`, который выполняет разогрев, серию повторных измерений, IQR-фильтрацию выбросов и вычисление устойчивых статистических характеристик, включая медиану, MAD и коэффициент вариации. Итоговое значение формируется как агрегированная оценка, менее чувствительная к случайным внешним воздействиям, чем единичный прогон.
8. **Реализован графический интерфейс, поддерживающий одновременное наблюдение и тестирование.** Пользователь получает структурированную среду с вкладками, карточками, индикаторами и панелями запуска бенчмарков. Важной особенностью стало то, что мониторинг и тесты сосуществуют в одном интерфейсе, что позволяет интерпретировать результаты не в отрыве от текущего состояния системы, а в связи с ним.
9. **Разработан интерактивный криптографический модуль.** Помимо предопределённых тестов, в системе реализована интерактивная

криптопанель, позволяющая выбирать алгоритм, параметры и объём данных. Это расширяет прикладную ценность комплекса и делает его полезным не только для статического сравнения платформ, но и для исследовательского анализа влияния параметров алгоритмов на производительность.

10. **Проведена экспериментальная проверка применимости разработанного комплекса.** В экспериментальной части показано, что предложенный подход позволяет организовать сопоставимое измерение на различных платформах, фиксировать контекст выполнения тестов и формулировать профильные выводы о сильных и слабых сторонах вычислительной системы. Было также продемонстрировано, что значимость имеет не только абсолютный результат, но и его устойчивость, а также связь с телеметрическим состоянием платформы.

Научно-методическая значимость работы заключается в том, что анализ вычислительной платформы рассматривается не как сумма независимых чисел, а как единый процесс сопоставления мониторинговых и измерительных данных. Подобный подход позволяет перейти от простой регистрации метрик к их осмысленной интерпретации. В условиях многозадачной операционной системы это особенно важно, поскольку числовой результат без сведений о фоне, разбросе и характере нагрузки может быть недостаточно информативным или даже вводящим в заблуждение.

Практическая ценность разработанного комплекса проявляется в нескольких направлениях. Во-первых, он может использоваться при первичной диагностике рабочей станции, ноутбука или сервера, когда требуется быстро выявить узкие места и оценить стабильность поведения системы. Во-вторых, комплекс пригоден для сравнительного анализа аппаратных платформ по различным профилям нагрузок, включая криптографические операции. В-третьих, он представляет интерес для учебного процесса, так как наглядно демонстрирует связь между архитектурой вычислительной системы, состоянием операционной среды и измеряемой производительностью. Наконец, комплекс может использоваться как вспомогательный инструмент при предварительной оценке пригодности платформы к задачам, чувствительным к памяти, вводу-выводу или криптографической нагрузке.

Отдельного внимания заслуживают ограничения проведённой работы. Несмотря на реализованную статистическую обработку и попытку контролировать условия измерений, полностью устранить влияние внешних факторов в пользовательской среде невозможно. Результаты зависят от текущего состояния операционной системы, особенностей планировщика, режимов энергосбережения, доступности аппаратных ускорителей и фоновой активности процессов. Кроме того, кросс-платформенность в прикладном смысле не означает абсолютной идентичности набора доступных метрик на всех ОС: часть телеметрических характеристик может предоставляться с различной степенью детализации или интерпретироваться по-разному. Осознание этих ограничений не снижает ценности работы, а подчёркивает её прикладной характер и методическую добросовестность.

Перспективы развития комплекса связаны с несколькими направлениями. Представляется целесообразным расширение телеметрии за счёт включения температурных датчиков, признаков троттлинга и иных показателей теплового состояния там, где это поддерживается платформой. Перспективным является введение специализированных профилей тестирования под конкретные сценарии эксплуатации, например серверного профиля, профиля разработчика, профиля аналитической станции или криптографического шлюза. Значительный потенциал имеет накопление и сравнение результатов в виде локального архива, что позволит анализировать изменение производительности системы во времени и выявлять деградации после обновлений или изменения конфигурации. Дополнительным направлением может стать включение тестов для оценки GPU, NPU и иных специализированных ускорителей, однако это потребует разработки отдельной методики и обеспечения сопоставимости результатов между платформами.

Итоговый вывод состоит в том, что цели и задачи, поставленные во введении, достигнуты. Разработан программный комплекс, который соответствует актуальным требованиям к инструментам системного анализа: он объединяет мониторинг и синтетическое тестирование, обеспечивает воспроизводимое измерение по нескольким категориям нагрузок, предоставляет пользователю интерпретируемые результаты и сохраняет архитектурную расширяемость. Созданное решение демонстрирует работоспособность выбранных проектных и программных подходов и подтверждает, что интеграция телеметрии и бенч-

маркинга в одном приложении является продуктивным направлением для задач прикладного анализа вычислительных систем.

Таким образом, выполненная работа находится на пересечении системного программирования, анализа производительности, прикладного бенчмаркинга и компьютерной безопасности. Полученный в её рамках программный комплекс не только фиксирует параметры функционирования вычислительной платформы, но и помогает интерпретировать их в контексте реальных сценариев эксплуатации. Это делает результаты анализа более содержательными, воспроизводимыми и практически полезными, а саму разработку — значимой как в инженерном, так и в учебно-исследовательском отношении.

Список использованных источников

Литература

- [1] Официальный сайт языка программирования Go [Электронный ресурс]. — URL: <https://go.dev/> (дата обращения: 21.12.2025).
- [2] Официальный сайт библиотеки Fyne [Электронный ресурс]. — URL: <https://fyne.io/> (дата обращения: 21.12.2025).
- [3] Донован, А. А., Керниган, Б. У. Язык программирования Go. — М.: Вильямс, 2016. — 496 с.
- [4] Shirou, G. gopsutil: библиотека для получения информации о системных процессах и использовании системы [Электронный ресурс] // GitHub. — URL: <https://github.com/shirou/gopsutil> (дата обращения: 21.12.2025).
- [5] Таненбаум, Э., Бос, Х. Современные операционные системы. 4-е изд. — СПб.: Питер, 2015. — 1120 с.
- [6] Паттерсон, Д., Хеннеси, Дж. Архитектура компьютера и проектирование компьютерных систем. 4-е изд. — СПб.: Питер, 2012. — 784 с.
- [7] Chan, A. Concurrency in Go: Tools and Techniques for Developers. — O'Reilly Media, 2017. — 256 p.
- [8] Sutter, H. The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software [Электронный ресурс] // Dr. Dobbs's Journal. — 2005. — URL: <http://www.gotw.ca/publications/concurrency-ddj.htm> (дата обращения: 21.12.2025).
- [9] Go Package Documentation: crypto, crypto/cipher, crypto/aes, crypto/rsa, crypto/ed25519 [Электронный ресурс]. — URL: <https://pkg.go.dev/crypto> (дата обращения: 31.03.2026).

- [10] Go x/crypto: дополнительные криптографические алгоритмы (в т.ч. ChaCha20-Poly1305) [Электронный ресурс] // GitHub. — URL: <https://github.com/golang/crypto> (дата обращения: 31.03.2026).
- [11] NIST FIPS PUB 180-4: Secure Hash Standard (SHS) [Электронный ресурс]. — URL: <https://csrc.nist.gov/publications/detail/fips/180/4/final> (дата обращения: 31.03.2026).
- [12] NIST FIPS PUB 197: Advanced Encryption Standard (AES) [Электронный ресурс]. — URL: <https://csrc.nist.gov/publications/detail/fips/197/final> (дата обращения: 31.03.2026).
- [13] NIST SP 800-56B Rev. 2: Recommendation for Pair-Wise Key Establishment Schemes Using Integer Factorization Cryptography [Электронный ресурс]. — URL: <https://csrc.nist.gov/publications/detail/sp/800-56b/rev-2/final> (дата обращения: 31.03.2026).
- [14] Tukey, J. W. Exploratory Data Analysis. — Addison-Wesley, 1977. — 688 p.
- [15] Hampel, F. R., Ronchetti, E. M., Rousseeuw, P. J., Stahel, W. A. Robust Statistics: The Approach Based on Influence Functions. — Wiley, 1986. — 536 p.
- [16] Hoaglin, D. C., Mosteller, F., Tukey, J. W. Understanding Robust and Exploratory Data Analysis. — Wiley, 1983. — 468 p.
- [17] Drepper, U. What Every Programmer Should Know About Memory [Электронный ресурс] // LWN.net. — 2007. — URL: <https://lwn.net/Articles/250967/> (дата обращения: 21.12.2025).
- [18] Intel 64 and IA-32 Architectures Optimization Reference Manual [Электронный ресурс] // Intel Corporation. — URL: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html> (дата обращения: 21.12.2025).
- [19] Geekbench - Cross-Platform Benchmark [Электронный ресурс]. — URL: <https://www.geekbench.com/> (дата обращения: 21.12.2025).

- [20] Go pprof - Profiling Go programs [Электронный ресурс] // GitHub. — URL: <https://github.com/google/pprof> (дата обращения: 21.12.2025).
- [21] Go Blog: Benchmarking in Go [Электронный ресурс]. — URL: <https://go.dev/blog/> (дата обращения: 31.03.2026).
- [22] gopsutil documentation: CPU, Memory, Disk, Net, Process [Электронный ресурс]. — URL: <https://pkg.go.dev/github.com/shirou/gopsutil/v3> (дата обращения: 31.03.2026).

Приложение А

Листинг точки входа приложения

(cmd/analyzer/main.go)

В приложении ниже приведён исходный код точки входа, отвечающей за инициализацию графического интерфейса и формирование основных вкладок комплекса.

```
1 package main
2
3 import (
4     "system-analyzer/pkg/gui"
5
6     "fyne.io/fyne/v2"
7     "fyne.io/fyne/v2/app"
8     "fyne.io/fyne/v2/container"
9 )
10
11 // main is the entry point of the application.
12 func main() {
13     // Create a new Fyne application.
14     a := app.New()
15     // Create the main application window with a title.
16     w := a.NewWindow("Системный Анализатор")
17
18     // Создаем контейнер с вкладками для различных панелей.
19     tabs := container.NewAppTabs(
20         container.NewTabItem("Мониторинг", container.NewScroll(gui.
21             NewDashboardPanel()))),
22         container.NewTabItem("Процессор", container.NewScroll(gui.
23             NewBenchmarkPanel("Процессор"))),
```

```

22     container.NewTabItem("Память", container.NewScroll(gui.NewBenchmarkPanel
    ("Память"))),
23     container.NewTabItem("Математика", container.NewScroll(gui.
    NewBenchmarkPanel("Математика"))),
24     container.NewTabItem("Криптография", container.NewScroll(gui.
    NewInteractiveCryptoPanel()))),
25 )
26
27 // Set the tab container as the main window content.
28 w.SetContent(tabs)
29 // Set window to maximum available size
30 w.Resize(fyne.NewSize(1600, 1000))
31 w.CenterOnScreen()
32 w.ShowAndRun()
33 }

```

Листинг А.1: Исходный код файла cmd/analyzer/main.go

Приложение В

Листинг обработчика измерений (pkg/benchmark/runner.go)

В данном приложении представлен ключевой компонент, обеспечивающий воспроизводимость результатов синтетических тестов. Функция `TestRunner` выполняет разогрев, серию итераций, фильтрацию выбросов и вычисление устойчивых статистик (медиана, MAD, коэффициент вариации).

```
1 package benchmark
2
3 import (
4     "math"
5     "sort"
6 )
7
8 type BenchmarkFunc func() (float64, string)
9
10 type BenchmarkStats struct {
11     Min      float64
12     Max      float64
13     Avg      float64
14     Median   float64
15     Unit     string
16     StdDev   float64
17     MAD      float64
18     Variance float64
19 }
20
```

```

21 func TestRunner(fn BenchmarkFunc, iterations int, progress chan<- float64)
    BenchmarkStats {
22     if iterations < 10 {
23         iterations = 10
24     }
25
26     // Паэорпс
27     _, _ = fn()
28     _, _ = fn()
29
30     results := make([]float64, iterations)
31     var unit string
32     for i := 0; i < iterations; i++ {
33         val, u := fn()
34         results[i] = val
35         unit = u
36         if progress != nil {
37             progress <- float64(i+1) / float64(iterations)
38         }
39     }
40
41     sort.Float64s(results)
42     median := calculateMedian(results)
43     mad := calculateMAD(results, median)
44     filtered := filterOutliersIQR(results)
45
46     var sum float64
47     min := filtered[0]
48     max := filtered[len(filtered)-1]
49     for _, v := range filtered {
50         sum += v
51     }
52     avg := sum / float64(len(filtered))
53     filteredMedian := calculateMedian(filtered)
54
55     var varianceSum float64
56     for _, v := range filtered {
57         diff := v - avg
58         varianceSum += diff * diff
59     }
60     stdDev := math.Sqrt(varianceSum / float64(len(filtered)))
61     cv := (stdDev / avg) * 100
62
63     finalResult := 0.6*filteredMedian + 0.4*avg
64     return BenchmarkStats{Min: min, Max: max, Avg: finalResult, Median:
        filteredMedian, Unit: unit, StdDev: stdDev, MAD: mad, Variance: cv}

```

Листинг В.1: Фрагмент файла pkg/benchmark/runner.go (ядро TestRunner)